BACHELOR'S THESIS / ENGINEERING 2025

CHASING BORDERS: COMPUTATIONAL OPTIMIZATION OF THE EUROPEAN MULTI-MODAL TRANSIT NETWORK FOR A GUINNESS WORLD RECORD ROUTING SEQUENCE

Aniol Petit Cabarrocas

upf. Universitat Pompeu Fabra Barcelona

Chasing Borders: Computational Optimization of the European Multi-Modal Transit Network for a Guinness World Record Routing Sequence

TREBALL DE FI DE GRAU DE

Aniol Petit Cabarrocas

Director: Jordi Taixés i Ventosa

Mathematical Engineering in Data Science

Curs 2025 - 2026

Universitat Pompeu Fabra Barcelona

Escola d'Enginyeria

Never tell me the odds

Agraïments

Tot i que el meu nom és el que presideix aquest treball, res del que llegireu a continuació existiria de la forma que ho fa sense l'ajuda i participació d'una sèrie de persones i institucions molt importants.

En primer lloc, vull agrair al CSUC per oferir el cluster i els serveis de computació gratuitament, i en especial a l'Iván Jiménez per ajudar-me a aconseguir més hores de computació després d'haver esgotat les que se m'havien assignat inicialment. Sense tot això no tindria ni una de les més de dos milions de rutes que he acabat trobant.

També vull agrair als que hi han estat en el dia a dia, però també als que eren més lluny i s'han assegurat que la distància no fos un impediment per a res. Mama, Papa, Clara, Núria, Avi, Jana, Aniol, Pol, Pau, Jan. Gràcies per escoltar-me divagar, queixar-me, celebrar i, en definitiva, per compartir aquest camí amb mi. Ha estat tot molt més fàcil i divertit amb vosaltres.

Jan, em semblava injust incloure't únicament a la llista d'abans considerant que aquest treball ni tan sols existiria sense tu i la teva percepció de la realitat brutalment alterada. Gràcies per ajudar-me a trobar la inspiració per fer aquest treball, però sobretot per recordar-me sempre que estem aquí per fer coses extraordinàries, a no conformar-me i a sempre anar a per més. Full gas we're never done!

I finalment, l'agraiment més especial és per tu, Jordi. El temps m'ha donat la raó amb la tria de tutor que vaig fer. M'has ajudat a pensar més enllà, a posar el fre i pensar abans de llençar-me a executar, a considerar opcions que ni se m'havien passat pel cap. En definitiva, m'has ajudat a fer el treball millor, i a la vegada a fer-me millor com a persona i com a professional. Qualsevol cosa que escrigui aquí es quedarà curta, per tant, simplement diré que mil gràcies.

Resum

Aquesta tesi aborda un problema d'optimització combinatòria amb un objectiu concret: determinar computacionalment la seqüència òptima de connexions de transport públic programat per batre el Rècord Mundial Guinness de més països visitats en un període estricte de 24 hores.
Per establir una base sòlida, la tesi construeix des de zero un conjunt de dades precís dels vols europeus i les connexions d'alta velocitat ferroviària, normalitzant inconsistències en la nomenclatura d'estacions de 40 països i sincronitzant tots els horaris a una referència UTC unificada. S'implementen i comparen dos paradigmes de grafs fonamentalment diferents: un model temporal Activity-on-Edge i un graf dirigit acíclic Activity-on-Node, aquest últim permetent distribuir una cerca en profunditat amb poda Branch-and-Bound en un clúster de computació d'alt rendiment de 100 nodes.
Tot i generar prop de dos milions d'itineraris candidats matemàticament vàlids, l'aplicació d'un Índex de Qualitat de Ruta personalitzat revela que la gran majoria depenen de connexions físicament impossibles. De 1.914.802 rutes candidates d'elit, exactament quatre sobreviuen com a seqüències viables per a un ésser humà. Els resultats estableixen que el límit físicament factible per a aquest intent de rècord és de 14 països, assolible mitjançant una de quatre seqüències específiques.

Abstract

This thesis addresses a combinatorial optimization problem with a concrete real-world objective: computationally determining the optimal sequence of scheduled public transport connections to break the Guinness World Record for the most countries visited within a strict 24-hour window.
To build a solid foundation, the thesis constructs a time-accurate dataset of European flights and high-speed rail connections from scratch, normalizing station naming inconsistencies across 40 countries and synchronizing all schedules to a unified UTC reference. Two fundamentally different graph paradigms are implemented and compared: an Activity-on-Edge time-dependent model and an Activity-on-Node Directed Acyclic Graph, the latter enabling a Depth-First Search with Branch-and-Bound pruning to be distributed across a 100-node High-Performance Computing cluster.
Despite generating nearly two million mathematically valid candidate itineraries, the application of a custom Route Quality Index reveals that the overwhelming majority rely on physically impossible connections. Out of 1,914,802 elite candidate routes, exactly four survive as human-viable sequences. The results establish that the physically feasible ceiling for this record attempt is 14 countries, achievable via one of four specific routing sequences.

Resumen

Esta tesis aborda un problema de optimización combinatoria con un objetivo concreto: determinar computacionalmente la secuencia óptima de conexiones de transporte público programado para batir el Récord Mundial Guinness de más países visitados en un período estricto de 24 horas.
Para establecer una base sólida, la tesis construye desde cero un conjunto de datos preciso de los vuelos europeos y las conexiones de alta velocidad ferroviaria, resolviendo inconsistencias en la nomenclatura de estaciones de 40 países y sincronizando todos los horarios a una referencia UTC unificada. Se implementan y comparan dos paradigmas de grafos fundamentalmente diferentes: un modelo temporal Activity-on-Edge y un grafo dirigido acíclico Activity-on-Node, este último permitiendo distribuir una búsqueda en profundidad con poda Branch-and-Bound en un clúster de computación de alto rendimiento de 100 nodos.
A pesar de generar casi dos millones de itinerarios candidatos matemáticamente válidos, la aplicación de un Índice de Calidad de Ruta personalizado revela que la gran mayoría dependen de conexiones físicamente imposibles. De 1.914.802 rutas candidatas de élite, exactamente cuatro sobreviven como secuencias viables para un ser humano. Los resultados establecen que el límite físicamente factible para este intento de récord es de 14 países, alcanzable mediante una de cuatro secuencias específicas.

Preface

Some of the best ideas arrive in the least convenient places.

It was the summer before my final year, somewhere on a beach in Sardinia, when Jan and I were keeping a football in the air with our heads. The kind of pointless, competitive thing you do when you have too much sun and too little agenda. At some point, he asked the question that tends to derail afternoons: what do you think the Guinness World Record is for this?

We didn't beat it. We weren't even close. But we did what we always do, which is start talking, and the conversation drifted (as conversations between us tend to) toward bigger, stranger records. That was when he mentioned this one. Most countries visited by scheduled public transport in 24 hours. He had known about it for a while, apparently. Something about it had stuck with him: the image of a single person threading themselves through the geography of an entire continent in a single day, boarding trains and planes with hard timings, and crossing borders that most people only see on maps.

I thought it was extraordinary. Not just as a physical challenge, but as a problem. Because underneath the adventure of it, the airports, the sprint connections and the border crossings, there was something that looked, to me, unmistakably like mathematics. A network. A clock. An objective function. And somewhere inside a combinatorial space too large to search by hand, an optimal sequence waiting to be found.

I had to choose my thesis topic within the week.

The decision took about ten minutes.

What followed was several months of building something that, to my knowledge, has never existed before: a unified, mathematically rigorous model of the European multi-modal transit network, engineered specifically to answer one question. Not approximately. Not heuristically. Exactly. Which sequence of scheduled flights and trains, departing on a real day, crossing real borders, with real transfer times that a real human being can actually execute, visits the most countries in 24 hours?

This thesis is the answer to that question. It is also, I hope, a demonstration that the right framing transforms an adventure into a science, and that sometimes the most interesting research problems arrive not from a reading list, but from a beach in Sardinia, a couple of beers and a friend who refuses to let a good question go unanswered.

Contents

AI Use Disclaimer

This thesis is the original work of myself (the author), developed with the strategic assistance of artificial intelligence (AI) tools in specific, defined capacities. Throughout the writing process, AI tools (specifically Gemini and Claude) were utilized to assist in redrafting select paragraphs to improve phrasing, academic tone, and overall readability, as well as to perform routine spelling and grammar checks. Despite this editorial assistance, the underlying arguments, analysis, and narrative structure of the text remain entirely my own.

In the technical phase of this project, AI was leveraged as an execution tool to generate the source code for the majority of the project files. This code generation was strictly instructional. All high-level architectural decisions, system design, logical sequencing, task prioritization, and underlying engineering reasoning were conceived and directed entirely by myself. The AI acted exclusively as a functional tool to implement explicit parameters and logic that I provided.

Finally, where AI technologies were directly used to produce the final artifacts, their specific implementation and operational roles are explicitly documented and explained in the appropriate technical sections of this thesis. Ultimately, all conceptual framework decisions, data interpretations, and conclusions drawn in this research are the sole responsibility of myself.

1 Introduction

The European continent, characterized by its high density of sovereign states and highly integrated, cross-border public transportation infrastructure, presents a unique theater for complex logistical optimization. This thesis tackles a highly constrained, multi-objective optimization problem framed by a distinct real-world application: computationally determining the global optimal routing sequence to break the Guinness World Record for the most countries visited by scheduled public transport within a strict 24-hour window (currently set at 13 countries). While the problem shares structural similarities with classical combinatorial problems such as the Orienteering Problem (OP) and the Travelling Salesman Problem with Time Windows (TSPTW), the existing literature on these problems is not directly applicable. The OP maximizes collected rewards along a path under a budget constraint, and the TSPTW minimizes travel cost while respecting node-level time windows; however, neither captures the defining characteristic of this problem: the objective function, the graph topology, and the feasibility constraints are all simultaneously governed by a discrete, real-world public transit timetable that cannot be abstracted away without losing the structure of the problem entirely. The optimization space is therefore specific enough to require a purpose-built algorithmic framework. The challenge requires maximizing the accumulation of unique sovereign territories while strictly minimizing temporal transit costs under a hard 24-hour deadline. Because transit connections (combining short-haul aviation and high-speed rail) do not exist continuously but are rigidly bound to specific departure and arrival schedules, traditional static spatial graphs are fundamentally inadequate. Modeling this dimension of time across a densely connected continental network introduces a massive combinatorial explosion, rendering standard brute-force enumeration computationally intractable.

To solve this, the research was structured across three major computational phases: data engineering, architectural comparative analysis, and distributed heuristic optimization. The first phase required the construction of a deterministic, time-accurate transportation dataset. Because no unified, free API exists for all European rail and flight networks, a data acquisition pipeline was engineered using a combination of flight REST APIs and programmatic web scraping of centralized rail aggregators. The resulting raw data underwent rigorous normalization to resolve naming inconsistencies, impute missing chronological boundaries, and map all localized schedules to a universally synchronized UTC timeline. To ensure the graph remained traversable, the network was geographically filtered using data-driven frequency heuristics and Large Language Models (LLMs) to isolate only the most critical international transit hubs, while specific intra-city transfers were manually validated to guarantee geospatial accuracy.

With the dataset normalized, the second phase focused on evaluating the optimal mathematical environment for the search algorithm. The thesis directly contrasts two fundamental paradigms for modeling time-dependent networks. The initial approach utilized an Activity-on-Edge (AoE) architecture, which maintained a highly compact memory footprint by compressing temporal schedules into dynamic arrays

on spatial edges. However, searching this graph required Dynamic Programming (DP) and the constant maintenance of expanding Pareto frontiers. This approach ultimately suffered a computational "time wall", choking on its own comparative loops and proving highly resistant to distributed parallelization due to Inter-Process Communication (IPC) latency. Consequently, the model was redesigned as an Activity-on-Node (AoN) framework. By transforming specific transit events into the graph's vertices, time was explicitly discretized, creating a strictly chronological Directed Acyclic Graph (DAG).

This structural pivot unlocked the final phase of the research: large-scale distributed optimization. The chronological nature of the AoN DAG decoupled the search space, allowing a Depth-First Search (DFS) integrated with Branch and Bound (B&B) pruning to be distributed across a 100-node High-Performance Computing (HPC) cluster. However, initial unconstrained executions revealed a blind spot: the algorithm's pursuit of mathematical minimums led it to exploit physically impossible transfers, which hijacked the B&B bounding logic and systematically pruned realistic, human-viable routes. To extract the true feasible optimum, a custom heuristic sorting mechanism was engineered to manipulate the algorithm's traversal stack, prioritizing realistic layover times over very short, physically impossible transfers. Finally, by applying a bespoke Route Quality Index (RQI) to score the physical viability of the generated permutations, the pipeline successfully filtered out millions of theoretical but infeasible routes to isolate the exact, physically executable global optimum, definitively establishing the theoretical and human-feasible limits of the European transit network.
The complete, reproducible Python codebase for data acquisition, graph construction, and distributed HPC execution developed for this research is publicly available at: `https://github.com/aniol-petit/route-planner-tfg.git`.

2 Theoretical Preliminaries

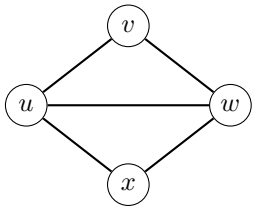
2.1 Graph Theory Foundations

To computationally solve a routing problem across the European transit network, the physical infrastructure must be translated into a mathematical structure known as a graph. A graph $G = (V, E)$ is defined by a set of vertices V (or nodes) and a set of edges E that connect pairs of vertices. In the context of transportation routing, a standard graph is inherently directed and weighted.

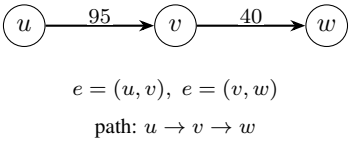
Directed edges. Transit connections are unilateral. An edge $e = (u, v)$ indicates a valid route from node u to node v. The existence of $e = (u, v)$ does not imply the existence of a return edge $e' = (v, u)$ at the same time or cost.

Weighted edges. Each edge carries a cost, typically represented by a weight function $w(u, v)$. In our context, this weight represents temporal costs, such as the travel duration of a flight or the waiting time of a layover.

A path in a graph is a sequence of vertices where each adjacent pair is connected by a valid edge. The goal of a routing algorithm is to find a specific path that maximizes or minimizes a certain objective function (e.g., minimizing total weight or maximizing the number of unique vertices visited).



(a) $G = (V, E)$ with $V = \{u, v, w, x\}$ and $E = \{(u, v), (v, w), \ldots\}$.



(b) $w(u, v) = 95$, $w(v, w) = 40$ (minutes); a routing algorithm searches for a path in G.

Figure 2.1: Panel (a) shows the abstract structure $G = (V, E)$; panel (b) shows directed edges $e = (u, v)$ and $e = (v, w)$ whose labels denote travel time in minutes, i.e. $w(u, v) = 95$ and $w(v, w) = 40$.

2.1.1 Time-Dependent vs. Time-Expanded Networks

A traditional static graph is insufficient for modeling public transit because connections do not exist continuously; they are strictly bound to specific departure and arrival schedules. To model this dimension of time, routing problems typically rely on one of two paradigms:

Time-Dependent Networks. The physical topology of the graph remains static (e.g., one vertex per station), but the weight of an edge $w(u, v, t)$ is a function of the departure time t. This keeps the network compact, but complicates the routing algorithms, as the cost of traversal fluctuates dynamically.
Time-Expanded Networks. Time is explicitly discretized and baked directly into the topology of the graph. A single physical station is duplicated into multiple vertices, each representing that station at a specific point in time (e.g., Station A at 10:00, Station A at 10:15). This significantly increases the number of nodes and edges, but allows the use of standard, static routing algorithms, since every edge now possesses a fixed, immutable weight.

Because our problem operates within a strict 24-hour window using discrete, fixed transit schedules, modeling time efficiently is the central architectural challenge. To systematically evaluate the optimal data structure for this Guinness World Record routing problem, this thesis intentionally implements and compares both paradigms: a memory-efficient time-dependent multi-graph (detailed in Chapter 4) and a structurally unrolled time-expanded network (detailed in Chapter 5). Similar comparisons have been made in the literature, see [1].

2.1.2 Activity-on-Edge (AoE) vs. Activity-on-Node (AoN)

Beyond the choice of how to handle chronological time, transportation graphs must also dictate where the transit data physically resides within the topology. This relies on two distinct structural paradigms:

Activity-on-Edge. The traditional approach where vertices represent physical locations, and edges represent the actual transport activities (e.g., a train moving between two cities) or waiting periods. The temporal data is stored as attributes on the edges.
Activity-on-Node. An inverted approach where the vertices themselves represent the specific transport activities (e.g., a flight with a fixed departure and arrival time), and the directed edges represent the valid layovers or transfer opportunities connecting them.

Understanding the distinction between AoE and AoN is critical, as the choice of paradigm radically alters the branching factor, memory footprint, and traversability of the search space. A core objective of this research is to contrast these two frameworks directly. Chapter 4 evaluates a Time-Dependent AoE structure, while Chapter 5 introduces a structural pivot to a Time-Expanded AoN framework, allowing for a comparative analysis of their computational viability on a continental scale.

2.2 Search and Optimization Algorithms

When graph networks become too large for brute-force enumeration, exact and heuristic optimization algorithms are required to traverse the search space efficiently.

2.2.1 Dynamic Programming and Label Setting

Dynamic Programming is a mathematical optimization method that solves complex problems by breaking them down into simpler, overlapping subproblems. In the context of graph routing, DP is often implemented via Label Setting algorithms. As the algorithm traverses the graph, it assigns "labels" to each node, where a label represents the accumulated cost or state of a specific path reaching that node.
In multi-objective optimization problems, where multiple conflicting criteria must be optimized simultaneously (e.g., minimizing time while maximizing profit), a single optimal path cannot always be determined dynamically. Instead, the algorithm must maintain a Pareto Frontier at each node. The frontier is a set containing all non-dominated labels. A label L_1 mathematically "dominates" L_2 if it is equal or superior in all evaluated objectives, and strictly superior in at least one. Dominated labels are permanently discarded, while the non-dominated set is kept to spawn future branches.

2.2.2 Depth-First Search (DFS) and Branch-and-Bound

DFS is a fundamental graph traversal algorithm that explores as far down a single branch of a tree as possible before backtracking. Because DFS only requires storing the nodes along the current active path, its memory complexity grows linearly with the depth of the search tree rather than exponentially with its breadth, making it highly memory-efficient.
To optimize DFS for finding absolute maximums or minimums, it is often paired with Branch-and-Bound, a paradigm for discrete and combinatorial optimization.

Branching. The algorithm systematically divides the search space by exploring subsequent child nodes.
Bounding. At each node, a bounding function calculates an optimistic mathematical estimate (an upper or lower bound) of the best possible solution that could be found by continuing down that branch.

If the calculated bound of a branch is worse than the best solution already found elsewhere in the search (the incumbent solution), the algorithm instantly abandons the branch. This technique, known as pruning, eliminates vast sections of the search space without requiring explicit evaluation.

2.3 Distributed Computing

When a computational problem's scale exceeds the memory or processing capacity of a single machine, distributed computing methodologies are employed to divide the task across an HPC cluster. An HPC cluster is a network of independent computers (nodes) that function as a single system. Access to these resources is governed by a workload manager or job scheduler, such as SLURM (Simple Linux Utility for Resource Management). The scheduler allocates specific hardware constraints, such as CPU cores and maximum RAM, and queues tasks to ensure that execution does not exceed physical memory limits, which would otherwise trigger immediate task termination.

To efficiently search massive combinatorial state spaces, algorithms frequently leverage data-level parallelism. In this paradigm, the exact same algorithm is executed concurrently across non-overlapping subsets of the initial data. In HPC environments, this is seamlessly orchestrated using job arrays. A job array takes a single execution script and duplicates it into dozens or hundreds of independent tasks, assigning each a unique environmental index. The algorithm uses this index to deterministically identify and process its specific slice of the search space, achieving massive throughput without the overhead of constant inter-node network communication.

Within those individual tasks, intra-node multiprocessing is often utilized to fully saturate the assigned CPU cores. However, launching multiple concurrent processes can lead to severe memory duplication, which is fatal for memory-intensive graph traversals. To mitigate this, concurrent architectures employ IPC and shared memory. By utilizing thread-safe locks and shared variables, multiple worker processes can safely read global data structures and update shared states, such as a global incumbent bound, without triggering race conditions or redundant memory allocation.

2.4 Data Acquisition Concepts

To populate large-scale routing networks, real-world scheduling data must be extracted programmatically from remote servers. The standard mechanism for this is a REST API (Representational State Transfer Application Programming Interface). APIs provide structured endpoints where a client can send HTTP requests to retrieve specific, filtered datasets. In modern web architectures, this data is almost universally exchanged in JSON (JavaScript Object Notation), a lightweight, key-value serialization format. JSON allows nested, hierarchical data to be seamlessly transmitted over networks and deserialized directly into programmatic data structures, such as dictionaries and arrays, for immediate computational use.

However, when public APIs are unavailable or heavily restricted, data must be gathered via web scraping. Web scraping involves automating HTTP requests to target servers and parsing the resulting HTML, a

markup language designed for human-readable browsers rather than machine consumption. This requires algorithmic extraction of target variables from unstructured or semi-structured Document Object Model (DOM) trees. Both API querying and web scraping are strictly governed by server-side rate limits (throttling). Consequently, data acquisition pipelines must implement pagination logic, exponential backoff strategies, and fault-tolerant parsing to ensure reliable, large-scale dataset generation without triggering server bans or connection timeouts.

3 Data Acquisition and Processing Pipeline

3.1 Sourcing Transit Data

The foundational step of this optimization problem was acquiring a comprehensive, time-accurate dataset of scheduled public transportation. The Guinness World Record guidelines mandate the use of scheduled public transport. Therefore, the data pipeline required a fusion of short-haul flight networks and high-speed rail schedules.

The geographic scope of the dataset was strictly and intentionally limited to the European continent. Europe is the only logical theater for this specific record due to its unparalleled density of sovereign states coupled with highly developed, cross-border transit infrastructure. However, extracting continental-scale transit data presents a significant financial barrier. Commercial aggregators typically charge per query, which is prohibitively expensive when building a dense, multi-country matrix. Consequently, a primary constraint of this project was to secure massive data volumes while strictly minimizing or eliminating acquisition costs. This budget constraint fundamentally dictated the technological approach for both flight and train data and imposed some challenges for acquiring high quality data.

Given the operational complexity and significant engineering overhead of the data acquisition pipeline described in this chapter, the dataset was extracted for a specific and fixed two-day window: January 22nd and 23rd, 2026. This date was selected as a representative standard weekday pair with no major public holidays across Europe, ensuring that flight and rail schedules reflect typical operational patterns rather than exceptional seasonal demand. Because the vast majority of scheduled European flights and high-speed rail services operate on recurring weekly timetables, this snapshot is considered a reliable proxy for the structural connectivity of the network. It is acknowledged, however, that any real-world Guinness World Record attempt would require repeating the full data acquisition and optimization pipeline for the specific dates of the attempt, as minor schedule variations, seasonal routes, and charter services could marginally affect the set of feasible routing sequences.

3.1.1 Flight APIs

While the airline industry is highly standardized, access to global distribution systems (such as Amadeus or Sabre) requires expensive commercial licensing. To bypass these costs, the flight dataset was sourced using the AviationEdge API (see API documentation in [2]).

This platform offered a generous 1 month trial period charged at $7 that enabled access to most of the endpoints, among those the *flightsHistory* endpoint used to acquire the flight data. To efficiently target the API without exhausting query limits on irrelevant locations, an online global dataset of airports (see

dataset in [3]) was acquired and programmatically filtered to isolate only active commercial airports within European borders. The resulting list of IATA (International Air Transport Association) codes served as the exact querying parameters.

By iterating through these European IATA codes and the specific 48-hour target date, the pipeline sent automated HTTP requests to extract all scheduled departures. The 48-hour window is required to ensure that routes starting at any point on day 1 have a full 24-hour window ahead of them to complete the record.

3.1.2 Train Data

Acquiring European rail data proved substantially more difficult than acquiring aviation data. Unlike the airline sector, which is strongly standardized around global distribution systems, European rail operations are fragmented across many national carriers (e.g., SNCF, DB, Renfe, and Trenitalia). No free, centralized API provides complete cross-operator schedules, while commercial rail datasets are prohibitively expensive for this project's budget.

As a result, the rail pipeline relied on programmatic extraction from the *Deutsche Bahn (DB) portal* (see website in [4]), which serves as a practical aggregator for a large share of international European rail connections. Because the DB interface is implemented as a modern Single-Page Application (SPA), data retrieval required browser-level automation rather than simple HTTP scraping.

Bot evasion and deterministic interaction. The scraper was implemented with *undetected-chromedriver* in a headless Selenium workflow. Standard Selenium configurations are commonly flagged by web application firewalls through automation fingerprints (e.g., `navigator.webdriver`). To reduce detection risk, the browser session was initialized with Chrome DevTools Protocol overrides that masked automation indicators and emulated normal browser characteristics.

The DB search UI also showed unstable behavior under keystroke-based automation because of dynamic autocomplete components and transient consent overlays. To improve robustness, form fields were populated through targeted JavaScript DOM injection (`driver.execute_script`), followed by explicit event dispatches (`change` and `blur`) so that the SPA state logic updated consistently.

Incremental time-sweeping for full-day coverage. Naïve pagination via "Later Connections" frequently triggered asynchronous DOM replacements and stale element references. To avoid these failures, the scraper used an incremental time-sweeping strategy: after parsing one result page, it captured the departure time of the last listed service, relaunched the query with that time as the new lower bound, and repeated this process until the full 24-hour window was covered.

Bi-directional corridor extraction and checkpointing. To constrain the immense scale of the European rail network and prevent the scraper from blindly querying thousands of unviable, low-speed rural combinations, the extraction was strictly limited to a predefined compilation of major international rail corridors sourced from train focused sites from the internet (see [5]). This curated list of high-speed routes (e.g., London to Paris, Munich to Budapest) was hardcoded into the pipeline to serve as the exact boundaries of the dataset. The extraction algorithm iterated through this specific list, parsed the strings containing interconnected stops, isolated the terminal origin and destination nodes, and executed the incremental scraper in both forward and reverse directions.

Given the computational cost and operational fragility of large-scale SPA extraction, the pipeline collected one representative weekday of rail schedules and duplicated that 24-hour block to form the 48-hour planning horizon required by the Guinness World Record routing formulation.

3.2 Data Cleaning and Normalization

With the raw flight and train schedules acquired, the data existed in highly disparate, semi-structured formats across thousands of CSV and JSON files. To safely process and mutate this data at scale, a

local SQLite database was instantiated as an intermediate staging environment. This approach enabled structured SQL queries to filter, group, and reconcile the massive datasets before exporting the finalized, clean records back to CSV format for the graph construction algorithm.

3.2.1 Entity Resolution and Standardization

A primary challenge in aggregating multi-modal European transit data is the absence of consistent naming conventions across transport networks. In practice, this meant that before a single route could be evaluated, the algorithm had to be taught that "München Hbf", "Munich Central", and "München Hauptbahnhof" were all the same physical place, and that "Frankfurt Flughafen" and "Frankfurt (Main) Hbf", despite sharing a city name, were emphatically not.

Toponymic normalization and multi-modal hub separation. To resolve this issue, a programmatic normalization pipeline mapped string variants to canonical entities. The algorithm first lowercased station text and then applied targeted heuristic matching to classify infrastructure types.

Because the optimization problem is explicitly multi-modal, normalization also had to distinguish transport modes within the same metropolitan area. For example, variants of “Frankfurt” were explicitly separated into the canonical rail hub “Frankfurt (Main) Hbf” and the aviation node “Frankfurt Flughafen”. Equivalent bifurcation rules were applied in Berlin (e.g., “Berlin Hbf” and “Berlin Südkreuz” versus “Flughafen BER”) and Munich.

The pipeline further unified strategically important cross-border junctions affected by strong linguistic variation. For example, heterogeneous labels around Basel and Lindau were merged into canonical station entities to preserve graph continuity at international interfaces.

Geopolitical entity standardization. Country labels also required strict harmonization between datasets. The aviation data provided ISO alpha-2 codes (e.g., “GB”, “MD”), which were mapped to textual country names using the `pycountry` library. However, these outputs did not always align with the naming used in rail records. Manual overrides were therefore introduced to prevent artificial country mismatches during route evaluation (e.g., mapping “United Kingdom” to “England”, and simplifying “Moldova, Republic of” to “Moldova”).

Deterministic temporal normalization. Although the flight API included historical and estimated delay attributes, these fields were removed from the final modeling dataset. Incorporating stochastic delay terms into a time-expanded graph substantially increases state-space complexity while offering limited predictive value for future schedules. The final normalized dataset therefore retained only scheduled departure and arrival times, preserving a deterministic DAG formulation.

3.2.2 The Midnight Rollover Problem and Absolute Minutes

To allow mathematical evaluation of layover times, all chronological data had to be converted into a continuous timeline. The target metric was *Absolute Minutes*, representing the elapsed time from $t = 0$ (midnight on the first day of the 48-hour routing window).

While flight records included explicit datetime strings, making baseline conversion straightforward, the scraped rail data provided mostly localized *HH:MM* values for intermediate stops. Because rail itineraries frequently cross midnight, this created a *Midnight Rollover* ambiguity: the raw strings did not indicate when the sequence advanced to the next calendar day, as the scraped data included only the times, not the dates. These were hardcoded in the scraping script, allowing us to inject them in the data manually and saving us from scraping another hard-to-extract field.

To resolve this, a sequential tracking algorithm reconstructed absolute time along each route. For each event, local time was converted to minute-of-day form:

$$M = \textit{hours} \times 60 + \textit{minutes}, \quad 0 \leq M < 1440.$$

Let t_{prev} denote the previously validated absolute timestamp and $M_{current}$ the current local minute value. The previous minute-of-day is:

$$M_{prev} = t_{prev} \bmod 1440.$$

A rollover flag is raised when the local clock wraps around:

$$R = \begin{cases} 1, & \text{if } M_{current} < M_{prev}, \\ 0, & \text{otherwise.} \end{cases}$$

With day index $D = \lfloor t_{prev}/1440 \rfloor$, the current absolute timestamp is computed as:

$$t_{abs} = (D + R) \times 1440 + M_{current}.$$

This formulation unfolds cyclic 24-hour clock values into a strictly increasing timeline suitable for graph operations. In parallel, a conservative imputation rule was applied to partial records: when arrival or departure values were missing, the available counterpart was copied to avoid `NULL`-driven discontinuities in the routing graph.

3.2.3 UTC Synchronization

Because Europe spans multiple time zones, locally normalized t_{abs} values are not directly comparable across borders. Without this step, the algorithm would perceive a train departing Madrid at 10:00 and a flight departing Lisbon at the same clock time as simultaneous events, when in reality the flight departs an hour earlier in absolute terms. A single unresolved timezone offset anywhere in the graph would silently corrupt every downstream routing decision that crossed that border.

To ensure strict chronological consistency, every absolute timestamp was projected onto a UTC+0 reference axis. Let Δ_{offset} denote the local UTC offset in minutes. The synchronized value is:

$$t_{UTC} = t_{abs} - \Delta_{offset}.$$

Determining Δ_{offset} robustly across heterogeneous nodes required a cascading lookup strategy:

- **Node-level overrides:** Explicit hardcoded mappings were applied first for exceptional nodes where country defaults are unreliable (e.g., Canary Islands $\Delta_{offset} = 0$, Azores $\Delta_{offset} = -60$, and eastern Russian hubs such as Yekaterinburg $\Delta_{offset} = 300$).
- **ISO-based country mapping:** If no node override existed, the ISO alpha-2 code was mapped to predefined regional offsets (e.g., Great Britain, Ireland, and Iceland to 0; Finland, Greece, and Moldova to 120).
- **Continental default:** Remaining nodes fell back to the Central European baseline $\Delta_{offset} = 60$.

This hierarchy provided a stable and computationally efficient timezone normalization layer for the full dataset.

After synchronization, a final integrity pass validated physical temporal feasibility for every leg:

$$t^{dep}_{UTC} < t^{arr}_{UTC}.$$

Any record violating this constraint was permanently removed, filtering upstream API inconsistencies and corrupted scrape artifacts from the final model.

3.3 Spatial Mapping and Filtering

To evaluate a Guinness World Record attempt based on the number of countries visited, the routing algorithm must inherently "know" the geographical location and sovereign territory of every single node in the network. Furthermore, a raw European transit graph contains tens of thousands of minor stops (e.g., suburban commuter stations or rural villages) that drastically bloat the search space without offering viable international transfer opportunities. Consequently, a rigorous spatial mapping and filtering pipeline was engineered to reduce the graph to its most mathematically relevant hubs while ensuring 100% geographical accuracy.

3.3.1 Country Assignment and Border Resolution

While international flights explicitly land in known countries, domestic and cross-border train schedules scraped from the DB portal do not consistently label the country of intermediate stops. To dynamically resolve this, a multi-strategy geographical detection algorithm was implemented, using tools like european train station datasets (see dataset in [6]).

The algorithm evaluated each scraped station name through a descending hierarchy of spatial heuristics:

- **Regex code extraction:** Extracting explicit country codes often hidden in parentheses within the raw text (e.g., parsing (CH) as Switzerland or (NL) as the Netherlands).
- **Language pattern recognition:** Utilizing linguistic markers to confidently infer sovereign territory (e.g., the presence of "hl.n." for Czech main stations, "Glowny" for Poland, or " b.Wien" for Austrian suburbs).
- **GeoNames cross-referencing:** Querying a localized subset of the GeoNames database (see dataset in [7]) to map normalized station strings to the most populous European city matching that name.

Despite these programmatic layers, complex border stations frequently generated false positives (e.g., "Genève" being administratively in Switzerland despite a French linguistic profile, or border towns like Domodossola). To guarantee absolute integrity for record evaluation, an exhaustive list of manual topological corrections was hardcoded into the pipeline to forcefully overwrite erroneous geographical assignments at critical border crossings.

3.3.2 Station Filtering and Hub Identification

With sovereign territories assigned, the extracted graph still remained computationally intractable. The raw network contained many low-value commuter stops, rural halts, and suburban nodes that inflated branching factor without improving continental routing viability. A dedicated pruning pipeline was therefore applied to remove irrelevant nodes while preserving the international backbone.

Lexical and topological pruning. The first pruning layer used a deterministic station-scoring heuristic. Each node was evaluated by (i) topological frequency (how often it appeared across scraped routes) and (ii) lexical indicators of infrastructure scale. The scorer rewarded high-capacity terms (e.g., "Hauptbahnhof", "Central", "Gare", "Aeropuerto") and penalized low-capacity markers (e.g., "Village", "Halt", "Suburban"). To prevent accidental graph fragmentation, terminal origin and destination stops of each train sequence were always retained, together with at least one high-scoring hub per traversed country.

LLM-assisted hub extraction. Although heuristic pruning removed most low-value nodes, identifying the most strategic international hubs across roughly 40 countries required additional geographic context. The pipeline therefore integrated Gemini 2.5 Pro as a constrained classifier rather than a routing engine (see [8] for information about the Gemini models family and their capabilities). National station lists were provided after heuristic filtering, and the model was prompted to return only internationally valuable hubs (e.g., cross-border high-speed access and strong air–rail interconnectivity). Outputs were constrained to

structured JSON for direct downstream use. This hybrid stage combined deterministic topological filtering with LLM-based geographic prioritization, yielding a compact graph concentrated around high-value transfer hubs.

3.3.3 Intra-City Transfers and Multi-Agent Geocoding Validation

The final spatial challenge was the one that no algorithm could solve on its own: connecting otherwise separate transport modalities within metropolitan areas. Getting from a flight arriving at Paris CDG to a train departing Paris Gare du Nord is not a graph problem, it is a human logistics problem, with rush-hour traffic, terminal sprawl, and the very real possibility that the connection simply cannot be made in time. Every intra-city transfer link in the dataset needed a travel time that a real person, carrying luggage, could actually execute. Getting this wrong would not produce a suboptimal route. It would produce a route that fails in the middle of the attempt.

Multi-agent LLM architecture. Directly estimating transfer feasibility through commercial geocoding APIs at continental scale was cost-prohibitive. To bypass this constraint, a two-agent workflow was implemented:

- **Generator agent:** evaluated hub pairs within the same country, proposed plausible intra-city connections, and estimated transit durations (local transit plus walking). A hard cutoff removed proposals with $\tau_{transfer} > 240$ minutes.
- **Auditor agent:** independently reviewed generated links, flagging implausible pairings and likely underestimates caused by geospatial hallucinations or naming ambiguities.

Ground-truth manual validation. The multi-agent stage reduced the candidate space to exactly 236 high-value intra-city transfer links. However, because LLM estimates remain probabilistic, a final manual validation pass was required for deterministic modeling quality. Each of the 236 links was manually verified in Google Maps, and exact walking/local-transit durations were extracted and hardcoded into the dataset. This final step eliminated subscription API costs while converting AI-proposed candidates into a physically viable, fully deterministic transfer matrix for routing optimization.

4

Algorithmic Approach I: Label Setting and Bounding Foundations

4.1 The Time-Dependent Transportation Graph

With a fully normalized, UTC-synchronized dataset of European transit schedules, the next step was constructing the mathematical environment for the routing algorithm. A standard static spatial graph $G = (V, E)$ is fundamentally inadequate for modeling scheduled public transportation, as an edge implies continuous availability.

To rigorously test different modeling paradigms, the first architectural approach evaluates the network as a time-dependent multi-graph. In this AoE architecture, the vertices remain purely spatial, while the dimension of time is encoded directly into the attributes of the edges. This approach was deliberately chosen to maintain a highly compact memory footprint, avoiding the node explosion inherent to fully time-expanded models.

The graph was instantiated using the NetworkX library (see library documentation in [9]) with the following structural rules:

- **Spatial Nodes (V):** A single vertex was created for each unique physical location. Nodes were tagged with specific attributes: their sovereign country (critical for the objective function) and their infrastructure type (airport or station).
- **Temporal Transit Edges (E):** Flights and trains were modeled as directed edges between origin and destination nodes. Because multiple trains or flights travel between the same two cities throughout the day, these edges stored dynamic temporal attributes. Instead of a single static weight, each edge contained an array of valid (`departure_time`, `arrival_time`) pairs, formatted in absolute UTC minutes. To account for the 48-hour Guinness World Record window, train schedules (which operate on daily cycles) were programmatically duplicated, adding 1440 minutes to populate the second day of the graph.
- **Static Transfer Edges:** The manually validated intra-city connections (e.g., traversing from a city's airport to its central rail station) were integrated as standard, bidirectional static edges. Unlike transit edges, transfer edges were assigned a fixed scalar weight representing the human connection time in minutes, as they can be traversed at any point during the day.

By compressing schedules into arrays on spatial edges, this AoE architecture successfully avoided node redundancy. However, this time-dependent structure shifts the chronological burden entirely onto the

search algorithm, which must dynamically iterate through edge arrays to find valid connections based on its arrival time at any given node. This compact graph serves as the baseline environment to test the exact optimization methods detailed in the following sections.

4.2 Algorithm Evolution: From Baseline to Bounding

Because the Guinness World Record challenge is fundamentally a multi-objective optimization problem (maximizing unique countries visited while minimizing time within a strict 24-hour limit) a standard shortest-path algorithm like Dijkstra's cannot be used. Instead, the search must be modeled using DP via a Label-Setting algorithm (see Algorithm 1 in Appendix B).

In this paradigm, as the algorithm traverses the time-dependent multi-graph, it assigns a state "label" L to each node. A label is defined by the tuple $L = (v, t, C, s)$. Here, v tracks the current node, t is the current absolute minute, C represents the mathematical set of unique countries visited so far, and s denotes `start_time`, which marks when the journey began. The cardinality of the set, denoted as $|C|$, represents the integer count of unique countries visited.

To prevent the search space from expanding infinitely, the algorithm relies on Pareto Dominance. At any given node, a new label L' is strictly dominated by an existing label L'' if the existing path arrived earlier or at the same time ($L''.t \leq L'.t$) while having visited a superset or equal set of countries ($L''.C \supseteq L'.C$), with at least one condition being strictly better. Dominated labels are instantly discarded.

However, implementing this baseline algorithm revealed a fundamental limitation: the Pareto dominance conditions were too relaxed. Across a network spanning roughly 40 countries, the combinatorial permutations of visited country sets C caused an explosion of non-dominated labels, rendering the baseline approach computationally unfeasible. This prompted a series of algorithmic refinements, where mathematical pruning techniques were progressively introduced to control the state space.

4.2.1 Reachability and Upper Bound (UB) Pruning

The first major architectural change was introducing predictive pruning (see Algorithm 2 in Appendix B). Before expanding a label, the algorithm calculated an optimistic Upper Bound (UB) of the maximum new countries that could theoretically be reached from the current node v within the remaining time.

To make this computationally tractable without running a full sub-search at every node, time was discretized into fixed intervals, or "buckets" b. For any given node v and time bucket b, a precomputation step (detailed in Algorithm 3, Appendix B) generated $R(v, b)$, the absolute set of all sovereign countries reachable from that location before the 24-hour clock expired. The theoretical Upper Bound was then calculated as the cardinality of the set difference between the reachable countries and the already visited countries: $UB = |R(v, b) \setminus C|$.

If the current number of countries visited plus this theoretical maximum ($|C| + UB$) was strictly less than or equal to the current global record, or couldn't beat the existing record, the path was mathematically dead and immediately pruned.

4.2.2 Time-Budgeted UB and Minimum Transit Time (MTT)

While standard UB pruning successfully removed hopeless paths, it was overly optimistic. It assumed that if 10 new countries were reachable from a node, all 10 could be visited, completely ignoring the physical time required to travel between them.

To tighten this bound, the concept of MTT was introduced. The MTT represents the absolute shortest physical time required to enter, traverse, and exit a specific country's transit network. For instance, even under perfect schedule conditions, traveling across Switzerland or Germany requires a mathematically

provable minimum number of minutes. The algorithm precomputed this MTT for every country in the dataset prior to the main search execution (see Algorithm 5 in Appendix B).

When calculating the UB, the algorithm sorted all unvisited, reachable countries from the fastest MTT to the slowest (see Algorithm 6 in Appendix B). It then sequentially added these minimum transit times to a `time_spent` counter. The moment the accumulated `time_spent` exceeded the actual hours remaining in the 24-hour journey, the algorithm stopped incrementing the UB. This "Time-Budgeted UB" improved convergence speed: it accurately pruned paths that possessed theoretical geographic connectivity but lacked the physical time budget required to actually execute the transits.

4.2.3 Priority Scoring, Hard Floors, and Rollouts

To further optimize the search, the standard queue was replaced with a Priority Queue. Rather than expanding blindly, labels were ranked using a custom scoring formula. The formula was designed to prioritize deep, fast routes. For example, a core heuristic evaluated $Score(L) = |C| \times 10000 - t$, effectively forcing the algorithm to expand labels with the highest country count, while using an earlier absolute time t as the mathematical tie-breaker to favor faster connections (see Algorithm 4 in Appendix B).

Despite prioritized expansion, execution logs revealed the algorithm still wasted resources expanding labels near the end of the timeline that had only accumulated a few countries. To counter this, a Global Hard Floor was introduced (see Algorithm 9 in Appendix B). By establishing a GlobalMinTime (the minimum realistic time to visit any single country), the algorithm calculated the physical time required to bridge the gap between $|C|$ and the target record. If this required time exceeded the remaining clock, the label was killed. Crucially, all arbitrary parameters for these heuristics (such as setting the GlobalMinTime to just 25 minutes) were deliberately defined to be overly optimistic. The architectural philosophy was to tolerate redundant computational overhead rather than risk pruning a mathematically viable, record-breaking route.

To capitalize on the Priority Queue, a Monte-Carlo Rollout mechanism was injected (see Algorithm 8 in Appendix B). Every 1,000 iterations, the algorithm paused standard expansion and launched a rapid, randomized "smart-greedy" probe deep into the graph (see Algorithm 7 in Appendix B). Using heavily biased weights to favor unvisited countries and a Tabu list to prevent local loops, these rollouts operated in milliseconds. If a rollout successfully reached the end of the day and beat the current global record, the new record was dynamically adopted, allowing the updated bound to prune a large portion of the priority queue immediately.

4.2.4 The Bi-directional Search Pivot

The final algorithmic evolution within this paradigm was motivated by the failure modes of the Monte-Carlo rollouts. Debugging revealed that rollouts were highly ineffective when they reached the night hours; physical train schedules became incredibly sparse, trapping the greedy probes in geographic dead-ends.

To bypass this nocturnal sparsity, the search was split into two halves. The architecture was rewritten as a Bi-directional Search: one algorithm ran forward in time from the start of the day, while an identical algorithm ran backward in time from the end of the day (see Algorithms 10 and 11 in Appendix B). Both halves executed for 14 hours, creating a 4-hour overlap in the middle of the day.

Finally, a dedicated "Stitcher" algorithm evaluated the intersection of these two sets (see Algorithm 12 in Appendix B). By iterating through overlapping nodes and validating time continuity and geographic unions ($C_{forward} \cup C_{backward}$), the stitcher could reconstruct a globally optimal 24-hour path. This parallel, bi-directional approach effectively bypassed the sparse night-time bottlenecks and maximized graph coverage during the dense daytime schedules.

4.3 The Dimensionality Collapse

Despite these successive refinements, from baseline Pareto dominance to time-budgeted Upper Bounds, hard floors, and bi-directional stitching, the architecture ultimately could not handle the combinatorial complexity. It is relevant to clarify that the algorithm did not fail due to an Out-Of-Memory (OOM) error. Rather, it hit a computational bottleneck.

The root of this bottleneck lay in the maintenance of the Pareto frontiers. As the search progressed deeper into the time-dependent multi-graph, the frontiers at each node grew increasingly dense. Because the Guinness World Record routing is fundamentally multi-objective (minimizing time versus maximizing unique countries), vast numbers of paths are mathematically incomparable. A route that visits 8 countries by 14:00 cannot dominate a route that visits 9 countries by 16:00. Consequently, both labels must be kept alive. As these non-dominated sets grew exponentially, the algorithm was forced to perform extensive comparative loops for every newly generated label just to establish dominance, making queue operations prohibitively expensive.

This inefficiency was heavily compounded by the execution environment. The algorithm was executed locally on a single machine rather than being deployed to an HPC cluster. This was not a limitation of resources, but a deliberate architectural necessity. Dynamic Programming and Label-Setting algorithms fundamentally rely on continually evaluating and updating a globally shared state, specifically, the dynamic Pareto frontiers at every node in the graph. Attempting to parallelize this specific architecture across distributed cluster nodes (which operate on isolated memory pools) would have introduced massive IPC overhead. The constant network latency required to safely lock and synchronize the frontiers across dozens of different parallel workers would likely have negated the benefits of parallelization. Thus, the approach was inherently bottlenecked by single-thread CPU execution speeds.

After approximately two weeks of continuous local execution across multiple algorithmic iterations, the best solution found by the AoE framework reached a maximum of 12 countries, falling short of the existing 13-country record. Crucially, this ceiling was not reached due to memory exhaustion but due to the computational cost of Pareto frontier maintenance, which grew prohibitively expensive as the search deepened. The inability to distribute this workload across parallel nodes, a direct architectural consequence of the shared global Pareto state, meant that no amount of additional wall-clock time could compensate for the single-thread bottleneck.

It is worth acknowledging that performance could theoretically have been pushed further. By implementing stricter dominance rules, more non-optimistic heuristics (such as aggressively killing technically viable paths to artificially force the queue to empty faster), or artificially forcing the queue to empty faster, the algorithm could have potentially achieved faster convergence. However, doing so would have completely sacrificed the exactness of the search. Given the highly specific, non-intuitive nature of train schedules, overly aggressive pruning risked the blind deletion of the optimal, record-winning route. More importantly, the ultimate objective of this research was not to infinitely tune a single algorithm, but to conduct a comparative evaluation between two fundamental modeling paradigms. After completing several major evolutionary iterations of the algorithm running on the AoE architecture, its core computational trade-offs were conclusively established: it is highly memory-efficient, but it shifts an excessive processing burden onto the search algorithm. The analysis now transitions to evaluating the second architectural candidate: the highly parallelizable AoN framework, explored in the following chapter.

5

Algorithmic Approach II: Graph-Based DFS Exploration

5.1 The Activity-on-Node (AoN) Transformation

As established in the preliminary definitions, a central objective of this research is to evaluate and contrast two fundamental graph paradigms for time-dependent routing. Having quantified the computational limits of the memory-efficient AoE architecture in Chapter 4, the analysis now shifts to the second structural candidate: the AoN framework. The AoE model prioritized a compact spatial topology, which inherently forced the search algorithm to dynamically calculate temporal validity at every node. To systematically test the inverse trade-off, the European transportation network was mathematically reconstructed as an AoN Directed Acyclic Graph (DAG). This approach encodes temporal constraints in the graph structure rather than in the search algorithm, prioritizing algorithmic traversal speed over memory footprint.

5.1.1 Structural Inversion: Events as Nodes

In the AoN paradigm, physical locations no longer exist as nodes. Instead, every single scheduled transport event (a specific flight or train) is instantiated as an independent vertex. A node in this graph is defined by the rigid tuple $v = (Origin, Destination, Departure\_Time, Arrival\_Time)$. For example, a flight leaving Barcelona at 10:00 and arriving in Rome at 12:00 represents one single, immutable node in the graph. By converting the temporal arrays stored on 8,456 spatial edges into individual nodes, the AoN transformation produced exactly 49,982 event nodes.

Consequently, the edges in the AoN graph no longer represent physical travel. Instead, directed edges represent valid transfer opportunities between two events. A directed edge from event node A to event node B is drawn if and only if:

- Event A's destination matches Event B's origin (or they are connected via a valid intra-city transfer).
- Event A's arrival time strictly precedes Event B's departure time, fully accounting for minimum physical transfer durations (e.g., $\text{Arrival}_A + \text{Transfer\_Time} \leq \text{Departure}_B$).

5.1.2 Trade-offs and the Directed Acyclic Guarantee

This inversion offered a key computational advantage: the search space became static. Because every edge in the AoN graph guarantees chronological validity, a DFS algorithm can traverse the network blindly. It does not need to verify temporal feasibility at each step; the existence of an edge guarantees a valid

transfer. Furthermore, because time strictly moves forward, the resulting graph is a DAG. It is physically impossible to traverse a loop, which naturally prevents infinite cycles without requiring complex Tabu lists.

However, this advantage comes with a severe structural trade-off: an exponential explosion in edge density. In a densely connected continental network, a single train arriving at a major hub like Frankfurt might theoretically connect to hundreds of subsequent departing trains. If every valid transfer opportunity is explicitly drawn as an edge, the branching factor of the DAG becomes prohibitively large for DFS traversal.

5.1.3 Earliest Arrival Pruning

To reduce this branching factor, we developed a domain-specific pruning heuristic during the graph construction phase: Earliest Arrival Pruning.

When evaluating all valid subsequent events (node B candidates) from a given arrival event (node A), the transformation algorithm grouped the candidates by their final destination. Rather than drawing edges to all valid subsequent trains heading to a specific city, the algorithm only drew a single edge to the train that offered the absolute earliest arrival time at that destination.

Mathematically, if Event A arrives in Munich at 14:00, and there are three valid subsequent trains to Vienna departing at 14:30, 15:00, and 16:00, creating edges to all three is redundant for a time-minimization objective. The 14:30 train dominates the others. By rigidly enforcing this pruning rule, tens of thousands of redundant temporal edges were removed during graph construction. Validation scripts confirmed this heuristic successfully reduced the total edge volume by over 75%, reducing the graph to approximately 1.5 million transfer edges representing the earliest-arrival connection to each destination.

5.2 Baseline DFS Execution on the Full Graph

With the European transit network successfully transformed into an AoN DAG and stripped of redundant temporal edges, the search space was structurally primed for a pure DFS. Unlike the Dynamic Programming approach evaluated in Chapter 4, which required maintaining continuously expanding Pareto frontiers in memory at every node, a DFS on a DAG only requires maintaining the current path stack. This exceptionally low memory footprint resolved the previous state-space memory bottlenecks, allowing the algorithm to trace routes with minimal memory overhead.

5.2.1 Core Search Mechanics and Bounding

At its conceptual core, the DFS algorithm is highly intuitive: it selects a valid starting node (e.g., a specific flight departing at 08:00) and exhaustively explores every possible sequence of connecting transports. The algorithm "walks" down a continuous path of flights and trains, accumulating countries along the way, until the 24-hour time limit expires. Once a path hits the deadline, the algorithm evaluates the total number of unique countries visited. If the total beats or ties the current global record, the route is saved. The algorithm then takes a step back (backtracks) to the previous station and tries the next available connecting branch, systematically exploring every single branch of the tree.

However, even on a highly pruned AoN graph, a pure brute-force traversal of a continental network would take centuries to execute. To control the exponential branching factor, the algorithm incorporated the mathematical bounding techniques developed in Chapter 4. The MTT and Upper Bound (UB) logic were seamlessly ported into the DFS as a B&B mechanism. At every step of the traversal, the algorithm calculated the time remaining before the 24-hour deadline, divided it by the MTT of the current country, and added this theoretical maximum to the current country count. If this optimistic Upper Bound could not mathematically beat the highest global record found thus far, the algorithm immediately abandoned that path and backtracked, avoiding unnecessary computation.

5.2.2 High-Performance Multiprocessing Architecture

Because a DFS evaluates independent branches, the search is an "embarrassingly parallel" problem. The architecture capitalized on this by gathering all valid starting nodes in the graph, defined as any transit event arriving early enough to allow a full 24-hour window, and distributing them across a pool of independent worker processes. Each worker was assigned a specific starting node and made solely responsible for exhaustively traversing all possible paths originating from that event.

To execute this at scale, the algorithm was deployed to an HPC cluster using the SLURM workload manager, allocating a single, highly powerful compute node equipped with 68 physical CPU cores and 64 GB of RAM.

Parallelizing a large graph search in Python, however, introduces non-trivial serialization and IPC overheads. To prevent these systems-level bottlenecks from significantly degrading the search speed, two critical engineering optimizations were implemented:

- **Zero-Serialization Memory Sharing:** Passing a 1.5 million-edge graph to 68 individual worker processes via standard Python multiprocessing would exhaust memory due to duplication overhead. Instead, the architecture utilized an initialization function (`init_worker`). This allowed the global graph dictionary, country mappings, and MTT data to be loaded natively into each worker's local memory space upon instantiation, avoiding repeated serialization.
- **Lazy Synchronization:** To allow all 68 workers to globally prune branches, they needed to share a single `shared_global_max` variable representing the current highest country record. Requiring all 68 workers to constantly acquire a multiprocessing lock to read this variable would cause significant IPC contention. To solve this, a "lazy sync" architecture was designed: each worker maintained a local copy of the global maximum and incurred IPC overhead only when syncronizing with the shared manager once every 10,000 nodes explored, or when a new record was actively discovered.

5.2.3 Live Checkpointing and the 48-Hour Bottleneck

To safeguard against cluster node failures or strict execution time limits, a fault-tolerant live-checkpointing system was integrated. As the 68 cores began traversing the graph, any path that successfully accumulated 14 or more countries, or tied the global maximum, was serialized and appended to a `live_checkpoint.json` file. This ensured that valuable, highly optimized candidate routes were continuously preserved to disk during execution without waiting for the entire algorithm to finish.

The algorithm was submitted to the cluster with a maximum wall-clock execution limit of 48 hours. Over the course of the run, the HPC node operated at maximum capacity, exploring billions of nodes and aggressively pruning unpromising branches.

However, when the 48-hour hard limit was reached and the cluster terminated the job, the execution logs revealed an unexpected outcome. Despite the extreme memory efficiency of the AoN DFS, the strict Branch and Bound pruning inherited from Chapter 4, and the deployment of 68 parallel cores, the algorithm had evaluated barely 1% of the total starting nodes in the graph. While the AoN transformation had successfully solved the memory issues of the previous approach, the unconstrained branching factor of the full, unfiltered continental network remained intractable within the time limit.

5.3 Graph Reduction and Targeted Executions

Although the 48-hour baseline DFS execution only evaluated 1% of the starting nodes in the full AoN graph, the live-checkpointing architecture yielded a valuable dataset. When the execution was terminated, the checkpoint file contained over 100,000 record-breaking candidate routes. This large number of successful paths, extracted from just 1% of the search space, suggested a hypothesis: while the algorithm

was struggling under the combinatorial weight of the full continental network, the actual winning routes were likely relying on the exact same core infrastructure.

To empirically test this hypothesis, a custom analytical dashboard was developed to parse the 100,000+ routes generated by the baseline run. The dashboard performed a rigorous Pareto analysis (frequency vs. cumulative percentage) on station and edge usage. The visual and statistical outputs conclusively confirmed the hypothesis: a small, elite subset of major international hubs and high-speed corridors was handling the vast majority of the routing traffic. Conversely, the dashboard's "long-tail" analysis revealed thousands of regional stations that appeared in almost zero winning permutations. The baseline algorithm was spending most of its time exploring regional stops that were irrelevant to any optimal solution for a continental speed record.

5.3.1 Data-Driven Graph Reduction

Based on this analysis, we shifted from exhaustive search to a targeted graph reduction. Rather than arbitrarily guessing which cities were important, the reduction was strictly data-driven. A pipeline was engineered to ingest the live checkpoint data, calculate the exact appearance frequency of every station across the 100,000 winning routes, and dynamically sever the low-value regional branches.
Using the original spatial graph as a baseline, two specific reduced subgraphs were generated:

- **The `freq100` graph (conservative):** A subgraph retaining only stations that appeared more than 100 times in the elite checkpoint routes.
- **The `freq1000` graph (aggressive):** A highly concentrated subgraph retaining only the absolute most critical arterial stations, requiring more than 1,000 appearances in the elite routes.

This pruning removed low-frequency regional stops, retaining only the hubs and corridors that appeared in optimal routes. These reduced spatial graphs were then transformed into new, significantly lighter AoN DAGs.

5.3.2 Full Convergence on Reduced Graphs

The identical 68-core DFS algorithm was subsequently deployed on these reduced AoN graphs. By mathematically severing the low-value regional branches, the algorithm was forced to strictly traverse the high-speed arterial core. The reduction in the branching factor yielded notable computational improvements.

Whereas the full graph had timed out after 48 hours while exploring just 1% of the space, the algorithm running on the aggressively pruned `freq1000` graph reached absolute completion, exhaustively evaluating 100% of all possible starting nodes and paths, in just 30 minutes. The conservatively pruned `freq100` graph, which retained a wider geographic net, successfully ran to absolute completion in exactly 3 hours.

By using the partial results of the initial run to prune the graph, the pipeline avoided the combinatorial blowup of the full search. The DFS was finally able to evaluate the entirety of the relevant temporal network, guaranteeing the extraction of the global optimums for the Guinness World Record attempt.

5.4 Distributed DFS Execution

Following the baseline DFS execution, logs indicated that the algorithm had evaluated only 1% of the valid starting nodes before hitting the 48-hour wall-clock limit. However, the nature of the AoN graph presented an opportunity: because each starting node represents an independent search tree, the problem can be inherently parallelized. Distributing the workload across 100 nodes should allow exhaustive evaluation within the same 48-hour window.

To execute this, the architecture was upgraded from a single-node multiprocessing script to a fully distributed SLURM Job Array (see cluster usage guide in [10]).

5.4.1 Horizontal Scaling and Job Array Distribution

The starting nodes were extracted, sorted deterministically, and partitioned across 100 independent array tasks. Rather than slicing the nodes into contiguous blocks, a round-robin distribution was implemented using modulo arithmetic (`index%total_jobs==job_idx`). This ensured that dense, high-traffic periods (e.g., morning rush hours) and sparse periods were evenly distributed across all 100 workers, ensuring no single worker was assigned a disproportionately dense portion of the schedule.

To further optimize the local traversal within each worker, a State Dominance Memoization technique was introduced. Each worker maintained a local hash map of previously visited states, defined by the tuple (`current_node`, `visited_countries`). If a worker arrived at a state it had already reached via an earlier or identical arrival time, the branch was instantly pruned.

5.4.2 OOM Hurdles and Distributed Checkpointing

Deploying 100 parallel jobs, each running multiple worker processes and maintaining local memoization dictionaries, placed an immense strain on the cluster's memory infrastructure. Early deployments of the distributed array encountered severe OOM errors, as the `local_visited_states` dictionaries grew beyond available RAM. To stabilize the array, the memory allocation per task had to be significantly increased (up to 64 GB per node) and the core count per task carefully balanced to ensure workers did not cannibalize shared memory.

Additionally, having 100 independent jobs concurrently attempting to write to a single, shared JSON checkpoint file would have caused I/O contention and potential file corruption. Instead, the checkpointing architecture was decentralized. Each job array task was assigned a unique, append-only JSONL (JSON Lines) file. As workers discovered routes visiting 14 or more countries, the paths were serialized as individual JSON objects and safely appended to their respective files, avoiding locking contention.

5.4.3 Route Deduplication and Parquet Compilation

Once the 100-node array completed its execution, the output consisted of 100 disparate JSONL files containing hundreds of thousands of candidate routes. Because the European transit network contains multiple converging paths, different starting nodes produced identical downstream sequences, resulting in significant duplication across output files.

To synthesize this fragmented output, a dedicated compilation pipeline was engineered. The compiler ingested every JSONL file and generated a deterministic MD5 hash for each route based on its sequence of specific transit events. This hashing mechanism successfully identified and dropped millions of redundant duplicate routes.

Finally, the deduplicated elite routes were flattened into a tabular structure and exported as a highly compressed `.parquet` file. This transition from nested JSON to columnar Parquet reduced the dataset's storage requirements and unlocked the ability to perform high-speed analytical queries. The resulting database contained all candidate routes visiting 14 or more countries, enabling the subsequent analysis and graph reduction.

5.5 Route Quality and Algorithmic Blind Spots

The distributed DFS successfully generated millions of candidate routes visiting 14 or more countries. However, raw mathematical validity does not perfectly align with human feasibility. Because the algorithm was strictly parameterized to maximize country count within 24 hours, it made use of the minimum-time

connections permitted by the graph. For example, the algorithm frequently selected "impossible" transfers, such as an itinerary arriving at a major airport and boarding a subsequent international flight departing just two minutes later. While mathematically valid within the strict confines of the AoN DAG, such a transfer is physically impossible for a passenger needing to traverse terminals, pass security, or manage gate closures.

5.5.1 The Route Quality Index (RQI)

To bridge the gap between mathematical optimality and physical reality, all candidate routes were subjected to a post-execution evaluation using a custom RQI. The RQI dynamically scores the physical feasibility of a route by evaluating the feasibility of every individual transfer, accounting for specific transportation modalities (airport "A" vs. rail/bus "R").

For each transfer, the engine calculates the available safety margin. The `raw_buffer` is the time between arrival and the next departure. From this, the algorithm subtracts the physical intra-city `transit_time` (pulled from the graph, or defaulting to 60 minutes if moving between different unmapped stations) and a modality-specific `warrior_minimum`. These minimums reflect the fastest realistic transit times used in the model: 20 minutes for A→A and A→R, 25 minutes for R→A (accounting for airport security), and 5 minutes for R→R (0 minutes if staying at the exact same rail station).
The effective safety margin is defined as

$$\texttt{margin} = \texttt{raw\_buffer} - \texttt{transit\_time} - \texttt{warrior\_minimum}. \tag{5.5.1}$$

Each transfer receives a piecewise quality score q based on this margin:

- **Non-negative margin** ($\texttt{margin} \geq 0$): Scored on an asymptotic exponential curve capped at 100:

$$q = 100\big(1 - e^{-0.05 \cdot \texttt{margin}}\big). \tag{5.5.2}$$

  A 30-minute safety buffer yields an excellent score (≈ 77), while anything above 90 minutes approaches 100.

- **Negative margin** ($\texttt{margin} < 0$): Heavily penalized with a linear multiplier:

$$q = \texttt{margin} \times 1000. \tag{5.5.3}$$

  A connection missing the minimum threshold by just 5 minutes receives a score of -5000.

Finally, because a single missed connection ruins the entire 24-hour attempt, the total RQI is calculated using a weighted formula that aggressively penalizes the route's weakest link (the bottleneck):

$$RQI = 0.7\, q_{\text{bottleneck}} + 0.3\, q_{\text{average}}. \tag{5.5.4}$$

By applying this formula, the dashboard successfully filtered out physically infeasible routes, allowing the generated itineraries to be ranked strictly by their real-world viability.

The weighting coefficients, modality-specific warrior minimums, and the exponential decay factor are acknowledged as engineering design choices rather than theoretically derived constants, and were decided according to my own experience and operational intuition, intentionally thought to be overly optimistic. The 70/30 bottleneck-to-average split reflects the operational reality that a single missed connection invalidates the entire attempt regardless of the quality of all other legs, making the weakest link disproportionately important.

5.5.2 The Missing Routes Paradox

During the cross-evaluation of the RQI-ranked routes, an unexpected inconsistency was discovered. When comparing the output of the conservatively pruned `freq100` graph against the output of the massive 100-node distributed array running on the full graph, some highly optimized, high-RQI routes present in the `freq100` dataset were entirely missing from the distributed dataset.

Logically, this should be impossible. The `freq100` graph is a strict subset of the full graph; therefore, any route found in the subset must mathematically exist in the superset. Since the distributed array was designed to exhaustively explore the full graph without missing any nodes, it should have theoretically captured every single route found by the `freq100` run, plus thousands more.

Investigation revealed a flaw in the interaction between the memoization logic and the full graph's branching factor. In the distributed worker script, a state was defined as (`current_node`, `visited_countries`). If the worker reached this state at an arrival time equal to or later than a previously recorded instance, the branch was pruned to save computational time.

However, because the algorithm was blindly iterating through the full graph's massive branching factor, it frequently reached these states via mathematically "fast" but physically absurd connections (e.g., 2-minute airport transfers). The worker would record this infeasible arrival time as the benchmark. Later, when the DFS naturally evaluated the more realistic, human-viable path (e.g., a path with a safe 90-minute layover), the algorithm would see that the arrival time was slower than the previously recorded route. As a result, the memoization logic pruned the feasible route, retaining only the physically impossible one.

5.5.3 Heuristic Re-Engineering and Stack Sorting

To counteract this, the search logic was re-engineered to force the DFS to prioritize realistic layovers over physically impossible short connections. To achieve this without altering the integrity of the exhaustive search, a heuristic sorting mechanism was injected directly into the graph dictionary initialization.

During graph construction, the algorithm evaluated all valid successors for a given node. The ideal human layover was defined as 90 minutes. This value was selected as the operational optimum balancing two competing pressures: it provides a meaningful safety buffer for a traveller executing a physically demanding international transit sequence, while avoiding excessively long waiting periods that would prematurely exhaust the 24-hour clock and reduce the maximum number of reachable countries. Layovers significantly shorter than this threshold are physically dangerous for the attempt; layovers significantly longer are strategically wasteful.

The successors were sorted in descending order (`reverse=True`) using an absolute-difference key:

$$\text{Sort Key} = \left|(\text{Departure}_{\text{next}} - \text{Arrival}_{\text{current}}) - 90\right|. \tag{5.5.5}$$

Because the DFS relies on a stack data structure, it operates on a Last-In, First-Out (LIFO) basis. This ordering had a direct mechanical effect:

- **Miracle and excessive transfers:** Layovers with extreme deviations from 90 minutes (e.g., a 2-minute transfer yielding a key of 88) were placed at the very front of the successor list, pushing them to the bottom of the DFS stack.
- **Optimal transfers:** Layovers close to 90 minutes (key near 0) were placed at the very end of the successor list, so they sat at the top of the DFS stack.

When the worker process called `stack.pop()`, it immediately extracted and explored the highly viable, $\sim$90-minute layovers first. This heuristic ordering forced the DFS to discover high-RQI routes early, setting realistic benchmark arrival times in the memoization dictionary so that feasible routes were not pruned.

5.5.4 Full Heuristic Execution and Convergence

With the heuristic sorting mechanism successfully integrated, the fully distributed AoN DFS was redeployed across the 100-node HPC array.

This heuristic change produced two benefits. Primarily, it prevented infeasible connections from overwriting the memoized arrival times of feasible routes. Secondarily, it also reduced the branching factor. By forcing the algorithm to actively consume the strict 24-hour global clock on realistic 90-minute layovers, individual paths accumulated fewer total transit legs before hitting the deadline. This naturally suppressed the combinatorial bloat that had previously caused the standard distributed search to balloon to nearly 3 million useless permutations.

As a direct result of these stabilized mechanics, the heuristic array achieved 100% graph convergence. Every assigned starting node across the unreduced continental network was exhaustively evaluated, with all 100 worker processes successfully completing their traversal well before the 48-hour SLURM time limit.

The disparate JSONL checkpoint files generated by the cluster were subsequently hashed, deduplicated, and flattened into a highly compressed columnar Parquet database. This finalized the execution phase of the project, successfully producing a database of over 1.9 million candidate routes visiting 14 or more countries. With the computational architecture fully realized and the theoretical graph exhaustively traversed, the analysis now transitions to Chapter 6 for the application of the RQI and the extraction of the true human-feasible global optimum.

6 Results and Conclusions

6.1 Computational Results and Execution History

The primary objective of this research, to computationally evaluate the topological limits of the European transit network and extract the global optimal routing sequence, was successfully achieved. Solving the problem required addressing successive computational bottlenecks, leading to five distinct search strategies. Analysis of these runs allows the theoretical and practical limits of the modeled network to be characterized.

6.1.1 The Evolution of the Search Architecture

To conquer the state-space explosion inherent in the AoN graph, the routing engine evolved through five major computational phases:

- **Baseline DFS (full graph):** A single-node multiprocessing approach. Due to the unconstrained branching factor, this exhaustive search evaluated merely 1% of the starting nodes before reaching the 48-hour wall-clock limit.
- **Topological reduction (`freq1000`):** A heavily pruned subgraph retaining only the absolute elite international hubs (those appearing in at least 1,000 routes from the Baseline DFS results). This search completed exhaustively in under an hour but missing routes that relied on less frequent transit lines.
- **Topological reduction (`freq100`):** A moderately reduced subgraph (nodes appearing in at least 100 routes from the Baseline DFS results). This search completed exhaustively in three hours, capturing a significant volume of viable paths but arbitrarily excluding lesser-known functional transit events.
- **Distributed DFS (full graph, non-heuristic):** A 100-node HPC array deployed across the full unreduced network. While it achieved 100% exploration, its blind chronological traversal caused the the memoization to record infeasible, minimum-time connections as the benchmark, overwriting realistic routes.
- **Distributed heuristic DFS (full graph):** The final architecture. By mathematically sorting successors to prioritize ideal 90-minute human layovers, this array achieved 100% exploration while protecting high-quality, feasible routes from being prematurely pruned by absurd permutations.

6.1.2 Comparative Yield and the Global Optimum

The outputs of these five executions were individually deduplicated and compiled into columnar Parquet databases for quantitative comparison. At this point in the pipeline, the combined output totalled nearly three million candidate routes, generated across hundreds of CPU-hours on a 100-node HPC cluster, each each satisfying the graph-level constraint of visiting 14 or more countries within 24 hours. The question was no longer whether the algorithm had found the optimal route. The question was how many of these millions of paths a human being could actually board. The metric for success was the discovery of "Elite Routes", defined as any sequence successfully visiting 14 or more sovereign countries within the 24-hour window. The answer is in Table 6.1.

Table 6.1: Comparative Analysis of Search Executions

Execution Strategy	**Search Space Explored**	**Elite Routes** (≥ 14)	**16-Country Routes**	**15-Country Routes**	**14-Country Routes**	**Routes with RQI** > 0
Baseline DFS	$\sim 1\%$ (Timeout)	114,158	53	3,625	110,480	0
Reduced (`freq100`)	100% (Reduced)	186,933	2,377	30,724	153,832	3
Reduced (`freq1000`)	100% (Reduced)	116,457	1,586	21,645	93,226	3
Distributed (Standard)	100% (Full)	2,901,144	3,658	128,361	2,769,125	0
Distributed (Heuristic)	100% (Full)	1,914,802	2,652	90,135	1,822,015	4

6.1.3 The Theoretical vs. Feasible Ceiling

As demonstrated in Table 6.1, the raw topology of the European network mathematically permits sequences visiting up to 16 countries. However, algorithmically valid paths do not inherently guarantee physical execution. When the RQI was applied to assess real-world human viability, evaluating transfer margins, minimum transit times, and modality constraints, every 15- and 16-country route yielded heavily negative scores. These paths relied on overlapping schedules and physically impossible transfers, proving them to be topological artifacts rather than executable itineraries.

When strictly filtering the final 1.9 million routes for positive viability ($\text{RQI} > 0$), the absolute feasible mathematical ceiling of the European transit network contracted definitively to 14 countries. Extracting these realistic paths required the full computational weight of the heuristic distributed array, which successfully isolated exactly four routing sequences capable of achieving this feasible global optimum.

The exact itineraries of these four routing sequences, including the specific flights, trains, departure times, and country sequences, are intentionally withheld from this publication. Given the direct competitive implications of this research for a live Guinness World Record attempt, disclosing the precise routes would compromise the exclusivity of the attempt prior to its execution.

6.2 Analysis of Results: The Gap Between Topology and Feasibility

The empirical data presented in Table 6.1 captures the fundamental tension at the core of this research: the vast discrepancy between what is mathematically permissible in a graph and what is physically executable in reality. The fluctuations in route volume, maximum country counts, and RQI scores across the five execution phases reveal critical insights into the behavior of the European transit network under extreme combinatorial constraints.

6.2.1 Network Density and the Pareto Principle (`freq100` vs. `freq1000`)

A direct comparison between the two reduced subgraph executions isolates the impact of network density on algorithmic yield. The `freq100` graph, being a larger superset containing thousands of additional secondary transit stations, mathematically generated a substantially higher volume of elite routes than the highly constrained `freq1000` graph (186,933 routes versus 116,457).

However, the application of the RQI reveals a strict Pareto distribution regarding physical feasibility. Despite the `freq100` execution evaluating exponentially more combinations, it yielded the exact same number of physically viable routes (3 routes with $\mathrm{RQI} > 0$) as the smaller `freq1000` graph. This explicitly demonstrates that the feasible, record-breaking capabilities of the European network are entirely reliant on a small fraction of major international hubs. Including secondary regional stations substantially increases search complexity without significantly improving the quality of feasible routes.

6.2.2 The B&B "Bullying" Phenomenon (Standard Distributed Execution)

A notable result from Table 6.1 is the output of the Standard Distributed search. Operating without constraints, it fully parsed the continent, yielding 2,901,144 elite routes. Yet, the physical viability of this dataset was zero.

This failure illustrates a flaw in applying standard B&B logic without feasibility constraints in real-world logistics. Because the standard search evaluated chronological connections indiscriminately, it readily utilized physically impossible transfers, such as 1-minute layovers between separate transport networks. These impossible connections allowed the algorithm to artificially pack a massive number of transit legs into the 24-hour window, quickly discovering mathematically valid 15- and 16-country paths early in its execution.

Once these anomalies were discovered, the B&B `global_max` variable was immediately raised to 15 or 16. From that point, the bounding logic efficiently pruned branches, but the inflated global maximum caused it to discard all physically realistic 14-country routes. Whenever the search evaluated a highly realistic, human-viable path (which typically necessitates 90-minute safety buffers), the B&B upper-bound calculation correctly determined that the path's maximum potential was capped at 14 countries. Because 14 is strictly less than the inflated `global_max` of 15 or 16, the algorithm pruned the branch. High-quality, perfect-RQI 14-country routes were systematically pruned because they could not match the inflated bound set by infeasible solutions.

Furthermore, the large output of 2.9 million routes was a direct consequence of these impossible transfers. A path built on 1-minute layovers can squeeze upwards of 30 distinct transport legs into a 24-hour window. This immense depth dramatically increases the graph's branching factor, producing a large number of combinatorial variants that do not correspond to executable itineraries.

6.2.3 Feasibility Extraction via the Heuristic Priority

The transition to the Distributed Heuristic execution pruning problem identified above and explains the final, definitive numbers of the study.

By restructuring the search to prioritize layovers closest to a 90-minute ideal, the algorithm fundamentally altered the timeline of discovery. It successfully mapped and logged the highly realistic 14-country routes before it eventually uncovered the 15- and 16-country physically impossible paths. Because the realistic routes were found prior to the `global_max` being artificially inflated, they were retained. This search ordering directly accounts for the successful extraction of the four optimal, human-viable routing sequences ($\mathrm{RQI} > 0$) out of the entire continental graph.

Equally significant is the reduction in total volume, dropping from 2.9 million to 1,914,802 elite routes. Unlike the standard search, the heuristic prioritization forced the algorithm to actively burn the 24-hour clock on realistic waiting periods. A path incorporating multiple 90-minute layovers consumes the

available 24 hours in far fewer total transit legs. This reduced depth naturally bounded the combinatorial branching factor, preemptively suppressing the explosion of useless permutations and resulting in a cleaner, vastly more concentrated final dataset.
Ultimately, this analysis proves that while the mathematical topology of the European transit network possesses the density to yield 16-country paths, the strict application of physical feasibility constraints definitively caps the absolute human optimum at 14 countries.

6.3 Methodological Verdict: AoE vs. AoN Paradigms

A primary objective of this research was to conduct a comparative evaluation of graph modeling paradigms for solving this highly constrained, multi-objective public transit routing problem. The transition from the time-dependent AoE architecture (Chapter 4) to the AoN DAG (Chapter 5) was not arbitrary, but rather motivated by distinct computational bottlenecks. The final results clearly identify the AoN paradigm as the more effective architecture for large-scale, exhaustive schedule evaluation.

6.3.1 The Failure of the Time-Dependent AoE Model

The AoE approach was initially selected for its memory efficiency. By representing transit stations as static vertices and compressing dynamic schedules into temporal arrays on the edges, the model successfully mapped the entire European transit network without triggering OOM errors.

However, this spatial compression shifted the entire burden of chronological calculation directly onto the search algorithm. Executing DP via a Label-Setting algorithm required maintaining Pareto dominance frontiers at every spatial node. The Guinness World Record constraint, minimizing time while maximizing distinct sovereign visits, created a scenario where vast numbers of paths were mathematically incomparable. A path that collected 8 countries by 14:00 could not mathematically dominate a path that collected 9 countries by 16:00.

Consequently, the non-dominated label sets at major transit hubs grew exponentially. The algorithm slowed to a halt, spending the majority of its CPU cycles executing comparative loops just to establish dominance for newly generated labels rather than exploring the search space.

Crucially, this architecture was not suited to distributed scaling. Because Label-Setting algorithms rely on constantly evaluating and updating shared global state (the dynamic Pareto frontiers), attempting to deploy this approach across an HPC cluster would have triggered massive IPC latency. The synchronization overhead required to lock and update frontiers across parallel workers would have neutralized any multi-core processing gains. Thus, the AoE approach was theoretically memory-efficient, but practically limited to single-thread execution, and failed to explore the search space within a practical time budget.

6.3.2 The Triumph of the AoN Directed Acyclic Graph

The transition to the AoN architecture required abandoning spatial compression. By transforming specific transit events (e.g., Train 123 from Paris to Brussels at 08:00) into the static nodes themselves, the graph exploded in memory footprint but gained a key structural property: it became strictly chronological, forming a DAG.
This change addressed the bottlenecks that made the AoE approach infeasible:

- **Elimination of the Pareto Loop:** In the AoN DAG, chronological time flows in one direction and paths never cyclically cross the exact same transit event. This eliminated the need to maintain expanding Pareto frontiers. Instead of comparing a new label against thousands of existing paths, the DFS algorithm only needed to perform a constant time ($O(1)$) dictionary lookup for state dominance against a single benchmark arrival time for that specific node and country set.

- **Decoupled Parallelization:** Because the AoN graph natively enforces chronological flow, it removed the algorithm's reliance on a dynamically shifting global state. The search space could be cleanly severed into discrete, independent starting nodes. This decoupling allowed the problem to be successfully deployed across a 100-node SLURM cluster. Each worker process navigated its assigned sector of the DAG completely independently, utilizing local memory dicts to execute state dominance pruning without ever triggering IPC latency.

Ultimately, the comparative study concludes that for multi-objective, time-constrained transit optimization over massive datasets, the memory-heavy AoN structure is strictly necessary. While the AoE approach was limited by the cost of Pareto comparisons, the AoN DAG's static structure enabled parallel DFS that explored millions of routes and identified the feasible optimum.

6.4 Conclusions and Future Work

6.4.1 Conclusions: The Feasible Optimum vs. Theoretical Topology

The primary objective of this thesis, to construct a comprehensive mathematical model of the European transit network and computationally extract the optimal routing sequence for the Guinness World Record, has been successfully accomplished. By transforming disparate transit schedules into a strictly chronological AoN DAG, and deploying a heuristic-driven distributed B&B search array, the theoretical bounds of the continent's infrastructure were definitively mapped.

The central finding of this research lies in the large gap between what is topologically possible in the graph and what is physically executable. The algorithm successfully generated nearly two million unique, mathematically valid itineraries capable of visiting 14, 15, or even 16 sovereign countries within 24 hours. However, the application of the RQI revealed that the vast majority of these topological pathways relied on overlapping schedules and impossible physical transfer margins.

After weeks of computation, architectural pivots, an architecturally constrained paradigm, a 100-node cluster, and nearly two million candidate itineraries, four routes satisfied the feasibility criteria.

Out of 1,914,802 mathematically elite paths, exactly four distinct routing sequences survived the RQI with a positive, human-viable safety margin. All remaining routes required physically impossible connections such as boarding a flight within minutes of landing or crossing between terminals without adequate transfer time. The European transit network is dense enough to support many theoretical routes, but the feasibility constraints narrow the viable options to a very small set. This research conclusively establishes that the absolute, physically feasible ceiling for this endeavor is 14 countries, and that achieving it requires following one of exactly four sequences. And it also demonstrates that graph-level optimality without feasibility constraints produces results that are mathematically valid but practically irrelevant.

6.4.2 Algorithmic Enhancements: The Layered Capping Search

While the heuristic distributed search successfully extracted the feasible global optimum, the interaction between the B&B pruning logic and the RQI presents a clear avenue for algorithmic optimization. As analyzed in Section 6.2, the aggressive nature of the B&B bounding function frequently pruned highly realistic, perfect-RQI 14-country routes because they were mathematically incapable of matching the physically impossible 15-country mathematical anomalies discovered earlier in the execution.

A primary direction for future algorithmic work is the implementation of a layered capping search. By intentionally capping the `global_max` bounding variable at the known human limit (14 countries), the algorithm would be forced to disable its aggressive upper-bound pruning for any branch capable of reaching that tier. This would effectively convert the algorithm from a global maximum search into an exhaustive enumerator for the 14-country space. While this would drastically increase the computational time required to evaluate that specific layer, it would guarantee the discovery and preservation of every

single high-RQI, human-viable route that was previously pruned out of the search space by inflated bounds from infeasible solutions.

6.4.3 Multi-Objective Attributes and Stochastic Modeling

Beyond graph traversal mechanics, the routing engine possesses the foundational architecture to support complex multi-objective optimization. The current RQI evaluates feasibility purely as a function of temporal transit margins. Future iterations of this engine could integrate additional dimensions, optimizing routes for minimal carbon footprint (CO_2 emissions per passenger kilometer), dynamic ticket pricing, or stochastic delay modeling.

Moving from a deterministic graph, where transit operates strictly on scheduled times, to a probabilistic graph would allow the algorithm to calculate the statistical likelihood of a route's success. However, while theoretically sound, integrating these dimensions requires access to highly restricted, fragmented, and often proprietary commercial datasets (such as historical airline delay APIs and dynamic rail pricing endpoints). Consequently, these enhancements were deliberately deferred to future work due to the inherent time constraints and data-access limitations of this academic scope, rather than algorithmic inability.

6.4.4 Broader Applications: A Pan-European Multi-Modal Engine

While the specific algorithm developed for this thesis targets a highly niche Guinness World Record application, the underlying architecture holds immense broader value. A significant contribution of this research is the data engineering pipeline: the successful parsing, UTC-synchronization, and topological normalization of disparate terrestrial feeds and aviation data into a unified DAG.

This unified data pipeline forms the core technological foundation required to build a comprehensive, consumer-facing multi-modal routing engine, effectively a "Google Maps" for integrated European transit. While flight aggregators and rail planners exist separately, integrated cross-modal engines remain an open engineering challenge. The architecture developed herein proves that such an integration is topologically possible and computationally navigable, and could serve as the foundation for future deployment via a Graphical User Interface (GUI) or commercial API.

6.4.5 Final Remarks

Through data normalization, architectural pivoting, graph modeling and distributed parallelization, the mathematical dimensions of the European transit network have been solved. The theoretical exploration of this routing problem is complete. The final remaining step in this endeavor is to step away from the computational model and attempt the physical execution of the optimal route, subject to receiving the necessary funding by interested sponsors. The logistical and adjudication frameworks for this physical Guinness World Record attempt are detailed in Appendix A.

Bibliography

[1] Pyrga E, Schulz F, Wagner D, Zaroliagis C. Efficient models for timetable information in public transportation systems. Journal of Experimental Algorithmics (JEA). 2007;12:Article 2.4. https://www.ceid.upatras.gr/webpages/faculty/zaro/pub/jou/J27-PSWZ-ACM-JEA-Vol12-N2.4-2008.pdf.

[2] Aviation Edge. Aviation Edge API Documentation;. https://aviation-edge.com/developers/.

[3] OurAirports. OurAirports Data;. https://ourairports.com/data/.

[4] Deutsche Bahn AG. Deutsche Bahn - International Bookings;. https://int.bahn.de/en/.

[5] Show Me The Journey. The International Rail Routes of Europe;. https://showmethejourney.com/travel-info-and-tips/the-international-rail-routes-of-europe/.

[6] Martin, Heads or Tails. Train Stations in Europe. Kaggle; 2020. https://www.kaggle.com/datasets/headsortails/train-stations-in-europe.

[7] GeoNames. GeoNames Geographical Database;. https://download.geonames.org/export/dump/.

[8] Google DeepMind. Gemini: a family of highly capable multimodal models. arXiv preprint arXiv:231211805. 2023;https://arxiv.org/pdf/2312.11805.

[9] Hagberg A, Swart P, S Chult D. Exploring network structure, dynamics, and function using NetworkX. In: Proceedings of the 7th Python in Science Conference; 2008. p. 11–15. https://scispace.com/pdf/exploring-network-structure-dynamics-and-function-using-2isdfk9y1h.pdf.

[10] Consorci de Serveis Universitaris de Catalunya (CSUC). Guia del docent per a l'ús de Pirineus;. https://confluence.csuc.cat/spaces/HPCKB/pages/392298538/Guia+del+docent+per+a+l%27us+de+Pirineus.

Appendices

The material in this part supplements the main text: Guinness World Record adjudication rules (Appendix A), Activity-on-Edge algorithm pseudocode (Appendix B), and data-normalization dictionaries (Appendix C).

A

Guinness World Record Adjudication Rules and Logistical Framework

The computational optimization developed in this thesis is strictly bounded by the physical and logistical rules set forth by Guinness World Records (GWR) for the attempt "Most countries visited in 24 hours by scheduled transport". To ensure the mathematical model produced a legal, record-eligible sequence, the algorithm was calibrated against the official guidelines and direct clarifications from the GWR Records Management Team.
The core rules and their direct impacts on the routing model are as follows:

- **Current record:** 13 countries.
- **Sovereign Definitions:** Eligible countries are strictly limited to the member states of the United Nations, Observer States, the Vatican, and Palestine. A "visit" is defined as physically setting foot within a country's borders, and each country is counted only once.
- **The "Country 0" Clock Initiation:** The GWR guidelines dictate that the continuous 24-hour clock starts the moment the challenger enters the first foreign country. The departure country does not count toward the total. Consequently, any initial domestic repositioning legs generated by the algorithm (e.g., a train to an airport within the starting country) were mathematically excluded from the final RQI country tally.
- **Transport Modalities:** The attempt must be executed using only scheduled transport available to the general public. However, the GWR team clarified that walking and paid, receipt-logged taxi journeys are permitted specifically for connecting between public transport hubs. This ruling validated the algorithm's use of static walking and local-transit times to bridge unconnected aviation and rail hubs within the same city.
- **Airport Sovereignty:** GWR confirmed that passing passport control at an airport constitutes a valid sovereign visit, even if the challenger does not leave the terminal building. This validated the inclusion of airport nodes as sovereign endpoints in the directed graph, provided the participant obtains a physical passport stamp and timestamped photographic evidence.
- **Evidence Requirements:** The physical execution of the optimal route requires continuous GPS tracking (timestamped `.kml` files), the chronological retention of all original transit tickets and receipts, hourly updated witness logbooks, and unbroken, static video evidence of the entire 24-hour attempt.

B

Label-Setting and Dynamic Programming Algorithms (AoE)

This appendix contains the formal pseudocode for the algorithms developed during the evaluation of the time-dependent AoE architecture discussed in Chapter 4. The algorithms are presented in their evolutionary order, demonstrating the progression from a naive baseline search to the final bi-directional stitching implementation.

This naive approach forms the foundational dynamic programming search, relying solely on standard Pareto dominance without predictive bounding.

Algorithm 1: Baseline label-setting algorithm for maximum country coverage

Input:

- V. Set of nodes
- E_s. Set of scheduled edges (v, u, dep, arr)
- E_t. Set of transfer edges (v, w, Δ)
- country(v). Country associated with node v

Output:

- P. Sequence of labels representing an optimal route within 24 hours

Initialize Labels$(v) \leftarrow \emptyset$ for all $v \in V$
Initialize queue $Q \leftarrow \emptyset$
Initialize best_label $\leftarrow$ null
// Initialize labels at departures
for all $v \in V$ **do**
    for all scheduled departure from v at time dep with arrival arr **do**
        $L \leftarrow (v, dep, \{\text{country}(v)\}, \text{pred} = \text{null}, \text{start\_time} = dep)$
        Labels$(v) \leftarrow$ Labels$(v) \cup \{L\}$
        $Q \leftarrow Q \cup \{L\}$
// Main loop
while $Q \neq \emptyset$ **do**
    extract $L = (v, t, C, \text{pred}, \text{start\_time})$ from Q
    // Only consider labels within 24-hour journey span
    if $t - L.\text{start\_time} > 24:00$ **then**
        continue
    if best_label $=$ null $\vee\ |C| > |\text{best\_label}.C|$ **then**
        best_label $\leftarrow L$
    // Extend via scheduled edges
    for all $(v, u, dep, arr) \in E_s$ **do**
        if $dep \geq t \wedge dep - L.\text{start\_time} \leq 24:00$ **then**
            $t_{\text{new}} \leftarrow arr$
            if $t_{\text{new}} - L.\text{start\_time} > 24:00$ **then**
                continue
            $C' \leftarrow C \cup \{\text{country}(u)\}$
            $L' \leftarrow (u, t_{\text{new}}, C', \text{pred} = L, \text{start\_time} = L.\text{start\_time})$
            if $\nexists L'' \in$ Labels(u) such that
              $(L''.t \leq t_{\text{new}} \wedge L''.C \supseteq C' \wedge (L''.t < t_{\text{new}} \vee L''.C \neq C'))$ **then**
                remove all $L'' \in$ Labels(u) such that
                  $(t_{\text{new}} \leq L''.t \wedge C' \supseteq L''.C \wedge (t_{\text{new}} < L''.t \vee C' \neq L''.C))$
                Labels$(u) \leftarrow$ Labels$(u) \cup \{L'\}$
                $Q \leftarrow Q \cup \{L'\}$
    // Extend via transfers
    for all $(v, w, \Delta) \in E_t$ **do**
        $t_{\text{new}} \leftarrow t + \Delta$
        if $t_{\text{new}} - L.\text{start\_time} > 24:00$ **then**
            continue
        $L' \leftarrow (w, t_{\text{new}}, C, \text{pred} = L, \text{start\_time} = L.\text{start\_time})$

```
36:         if ∄L'' ∈ Labels(w) such that
                (L''.t ≤ t_new ∧ L''.C ⊇ C ∧ (L''.t < t_new ∨ L''.C ≠ C)) then
37:             remove all L'' ∈ Labels(w) such that
                    (t_new ≤ L''.t ∧ C ⊇ L''.C ∧ (t_new < L''.t ∨ C ≠ L''.C))
38:             Labels(w) ← Labels(w) ∪ {L'}
39:             Q ← Q ∪ {L'}
40: // Reconstruct path
41: P ← empty sequence
42: L ← best_label
43: while L ≠ null do
44:     prepend L to P
45:     L ← L.pred
46: return P
```

Introduces predictive pruning using precomputed reachable country sets to eliminate hopeless branches.

Algorithm 2: Label-setting algorithm with UB pruning for maximum country coverage

Input:

- V. Set of nodes
- E_s. Set of scheduled edges (v, u, dep, arr)
- E_t. Set of transfer edges (v, w, Δ)
- country(v). Country associated with node v
- $R(v, b)$. Precomputed reachable country sets
- bucket(t). Time bucket mapping function

Output:

- P. Sequence of labels representing a route within 24 hours

Initialize Labels$(v) \leftarrow \emptyset$ for all $v \in V$
Initialize queue $Q \leftarrow \emptyset$
Initialize best_label $\leftarrow$ null
// Initialize labels at departures
for all $v \in V$ **do**
    for all scheduled departure from v at time dep with arrival arr **do**
        $L \leftarrow (v, dep, \{\text{country}(v)\}, \text{pred} = \text{null}, \text{start\_time} = dep)$
        Labels$(v) \leftarrow$ Labels$(v) \cup \{L\}$
        $Q \leftarrow Q \cup \{L\}$
// Main loop
while $Q \neq \emptyset$ **do**
    extract $L = (v, t, C, \text{pred}, \text{start\_time})$ from Q
    // Enforce 24-hour journey limit
    if $t - \text{start\_time} > 24:00$ **then**
        continue
    // Upper bound pruning
    $b \leftarrow \text{bucket}(t)$
    $\text{UB} \leftarrow |R(v, b) \setminus C|$
    if best_label $\neq$ null $\wedge\ |C| + \text{UB} \leq |\text{best\_label}.C|$ **then**
        continue
    // Update best label
    if best_label = null $\vee\ |C| > |\text{best\_label}.C|$ **then**
        best_label $\leftarrow L$
    // Extend via scheduled edges
    for all $(v, u, dep, arr) \in E_s$ **do**
        if $dep \geq t \wedge dep - \text{start\_time} \leq 24:00$ **then**
            $t_{\text{new}} \leftarrow arr$
            if $t_{\text{new}} - \text{start\_time} > 24:00$ **then**
                continue
            $C' \leftarrow C \cup \{\text{country}(u)\}$
            $L' \leftarrow (u, t_{\text{new}}, C', \text{pred} = L, \text{start\_time} = \text{start\_time})$
            if $\nexists L'' \in$ Labels(u) such that $(L''.t \leq t_{\text{new}} \wedge L''.C \supseteq C' \wedge (L''.t < t_{\text{new}} \vee L''.C \neq C'))$ **then**
                remove all $L'' \in$ Labels(u) such that $(t_{\text{new}} \leq L''.t \wedge C' \supseteq L''.C \wedge (t_{\text{new}} < L''.t \vee C' \neq L''.C))$
                Labels$(u) \leftarrow$ Labels$(u) \cup \{L'\}$

```
    Q ← Q ∪ {L′}
  // Extend via transfers
  for all (v, w, Δ) ∈ E_t do
    t_new ← t + Δ
    if t_new − start_time > 24 : 00 then
      continue
    L′ ← (w, t_new, C, pred = L, start_time = start_time)

    if ∄L″ ∈ Labels(w) such that
        (L″.t ≤ t_new ∧ L″.C ⊇ C ∧ (L″.t < t_new ∨ L″.C ≠ C)) then
      remove all L″ ∈ Labels(w) such that
          (t_new ≤ L″.t ∧ C ⊇ L″.C ∧ (t_new < L″.t ∨ C ≠ L″.C))
      Labels(w) ← Labels(w) ∪ {L′}
      Q ← Q ∪ {L′}
// Reconstruct path
P ← empty sequence
L ← best_label
while L ≠ null do
  prepend L to P
  L ← L.pred
return P
```

The precomputation step required for Algorithm 2. It calculates the $R(v, b)$ sets (reachable countries from node v at time bucket b) via backward propagation.

Algorithm 3: Relaxed reachability backpropagation

Input:

- V. Set of nodes
- E_s. Set of scheduled edges (v, u, dep, arr)
- E_t. Set of transfer edges (v, w, Δ)
- country(v). Country associated with node v
- bucket(t). Time bucket mapping function
- $B_{\max}$. Maximum bucket index within 24-hour horizon

Output:

- $R(v, b)$. Set of reachable countries for all $v \in V$, $b \in \{0, \ldots, B_{\max}\}$

// Initialize reachability sets
for all $v \in V$ **do**
    for $b = 0$ to $B_{\max}$ **do**
        $R(v, b) \leftarrow \{\text{country}(v)\}$
// Build relaxed adjacency lists
Initialize $\text{Adj}(v, b) \leftarrow \emptyset$ for all $v \in V$, $b \in \{0, \ldots, B_{\max}\}$
// Scheduled edges
for all $(v, u, dep, arr) \in E_s$ **do**
    $b \leftarrow \text{bucket}(dep)$
    $b' \leftarrow \text{bucket}(arr)$
    if $0 \le b \le B_{\max} \wedge 0 \le b' \le B_{\max}$ **then**
        $\text{Adj}(v, b) \leftarrow \text{Adj}(v, b) \cup \{(u, b')\}$
// Transfer edges
for all $(v, w, \Delta) \in E_t$ **do**
    for $b = 0$ to $B_{\max}$ **do**
        $t \leftarrow \text{start\_time\_of\_bucket}(b)$
        $t' \leftarrow t + \Delta$
        $b' \leftarrow \text{bucket}(t')$
        if $0 \le b' \le B_{\max}$ **then**
            $\text{Adj}(v, b) \leftarrow \text{Adj}(v, b) \cup \{(w, b')\}$
// Backward propagation over time buckets
for $b = B_{\max}$ downto 0 **do**
    repeat
        changed $\leftarrow$ false
        for all $v \in V$ **do**
            for all $(u, b') \in \text{Adj}(v, b)$ **do**
                if $R(u, b') \setminus R(v, b) \neq \emptyset$ **then**
                    $R(v, b) \leftarrow R(v, b) \cup R(u, b')$
                    changed $\leftarrow$ true
    until changed = false
return R

Replaces the standard queue with a priority queue, scoring labels based on country count and time to aggressively expand promising nodes.

Algorithm 4: Label-setting algorithm with UB pruning, scoring, and early stopping

---

In this initial version, we use the following scoring formula:
$Score(L) = |C| + (1 - \tau)UB$; where $\tau = \frac{t - \text{start\_time}}{24:00}$

Input:

- V. Set of nodes
- E_s. Set of scheduled edges (v, u, dep, arr)
- E_t. Set of transfer edges (v, w, Δ)
- country(v). Country associated with node v
- $R(v, b)$. Precomputed reachable country sets
- bucket(t). Time bucket mapping function
- record. Current record to beat (e.g. 13)

Output:

- P. Sequence of labels representing a route within 24 hours

Initialize Labels$(v) \leftarrow \emptyset$ for all $v \in V$
Initialize priority queue $Q \leftarrow \emptyset$ // ordered by decreasing $score, |C|, UB, -t$
Initialize best_label $\leftarrow$ null
// Initialize labels at departures
for all $v \in V$ **do**
    for all scheduled departure from v at time dep with arrival arr **do**
        $L \leftarrow (v, dep, \{\text{country}(v)\}, \text{pred} = \text{null}, \text{start\_time} = dep)$
        Labels$(v) \leftarrow$ Labels$(v) \cup \{L\}$
        compute score(L)
        $Q \leftarrow Q \cup \{L\}$
// Main loop
while $Q \neq \emptyset$ **do**
    extract $L = (v, t, C, \text{pred}, \text{start\_time})$ with maximum score from Q
    // Enforce 24-hour journey limit
    if $t - \text{start\_time} > 24:00$ **then**
        continue
    // Upper bound computation
    $b \leftarrow$ bucket(t)
    UB $\leftarrow |R(v, b) \setminus C|$
    // Record-based pruning
    if $|C| + \text{UB} \leq$ record **then**
        continue
    // Update best label
    if best_label $=$ null $\vee\ |C| >$ |best_label.C| **then**
        best_label $\leftarrow L$
        if $|C| >$ record **then**
            break // early stopping: record beaten
    // Extend via scheduled edges
    for all $(v, u, dep, arr) \in E_s$ **do**
        if $dep \geq t \wedge dep - \text{start\_time} \leq 24:00$ **then**
            $t_{\text{new}} \leftarrow arr$
            if $t_{\text{new}} - \text{start\_time} > 24:00$ **then**
                continue

$C' \leftarrow C \cup \{\text{country}(u)\}$
$L' \leftarrow (u, t_{\text{new}}, C', \text{pred} = L, \text{start\_time} = \text{start\_time})$

if $\nexists L'' \in \text{Labels}(u)$ such that
$(L''.t \leq t_{\text{new}} \wedge L''.C \supseteq C' \wedge (L''.t < t_{\text{new}} \vee L''.C \neq C'))$ **then**
remove all $L'' \in \text{Labels}(u)$ such that
$(t_{\text{new}} \leq L''.t \wedge C' \supseteq L''.C \wedge (t_{\text{new}} < L''.t \vee C' \neq L''.C))$
$\text{Labels}(u) \leftarrow \text{Labels}(u) \cup \{L'\}$
compute score(L')
$Q \leftarrow Q \cup \{L'\}$

// Extend via transfers
for all $(v, w, \Delta) \in E_t$ **do**
$t_{\text{new}} \leftarrow t + \Delta$
if $t_{\text{new}} - \text{start\_time} > 24:00$ **then**
continue
$L' \leftarrow (w, t_{\text{new}}, C, \text{pred} = L, \text{start\_time} = \text{start\_time})$

if $\nexists L'' \in \text{Labels}(w)$ such that
$(L''.t \leq t_{\text{new}} \wedge L''.C \supseteq C \wedge (L''.t < t_{\text{new}} \vee L''.C \neq C))$ **then**
remove all $L'' \in \text{Labels}(w)$ such that
$(t_{\text{new}} \leq L''.t \wedge C \supseteq L''.C \wedge (t_{\text{new}} < L''.t \vee C \neq L''.C))$
$\text{Labels}(w) \leftarrow \text{Labels}(w) \cup \{L'\}$
compute score(L')
$Q \leftarrow Q \cup \{L'\}$

// Reconstruct path
$P \leftarrow$ empty sequence
$L \leftarrow$ best_label
while $L \neq$ null **do**
prepend L to P
$L \leftarrow L.\text{pred}$
return P

Calculates the absolute minimum time required to enter and exit each specific country's transit network to enable time-budgeted pruning.

Algorithm 5: Minimum Transit Time (MTT) Precomputation

Input:

- V. Set of nodes
- E_s. Set of scheduled edges (v, u, dep, arr)
- E_t. Set of transfer edges (v, w, Δ)
- country(v). Country associated with node v

Output:

- MTT. Map of each country to the absolute minimum time required to enter it

Initialize $MTT(c) \leftarrow \infty$ for all $c \in$ unique countries
// Process scheduled edges
for all $(v, u, dep, arr) \in E_s$ **do**
    if country$(v) \neq$ country(u) **then**
        transit_time $\leftarrow arr - dep$
        if transit_time $< MTT($country$(u))$ **then**
            $MTT($country$(u)) \leftarrow$ transit_time
// Process transfer edges (e.g., cross-border footpaths or local transit)
for all $(v, w, \Delta) \in E_t$ **do**
    if country$(v) \neq$ country(w) **then**
        if $\Delta < MTT($country$(w))$ **then**
            $MTT($country$(w)) \leftarrow \Delta$
return MTT

Integrates the MTT precomputation (Algorithm 5) into the bounding logic to enforce physical time constraints on the theoretically reachable countries.

Algorithm 6: Label-setting algorithm with time-budgeted UB pruning, scoring, and early stopping

In this initial version, we use the following scoring formula:
$Score(L) = |C| + (1 - \tau)UB$; where $\tau = \frac{t - \text{start\_time}}{1440}$

Input:

- V. Set of nodes
- E_s. Set of scheduled edges (v, u, dep, arr)
- E_t. Set of transfer edges (v, w, Δ)
- country(v). Country associated with node v
- $R(v, b)$. Precomputed reachable country sets
- $MTT(c)$. Precomputed minimum transit times for each country
- bucket(t). Time bucket mapping function
- record. Current record to beat (e.g. 13)

Output:

- P. Sequence of labels representing a route within 24 hours

Initialize Labels$(v) \leftarrow \emptyset$ for all $v \in V$
Initialize priority queue $Q \leftarrow \emptyset$ // ordered by decreasing score
Initialize best_label $\leftarrow$ null
// Initialize labels at departures
for all $v \in V$ **do**
    for all scheduled edges $(v, u, dep, arr) \in E_s$ **do**
        $L \leftarrow (u, arr, \{\text{country}(u)\}, \text{pred} = \text{null}, \text{start\_time} = dep)$
        Labels$(u) \leftarrow$ Labels$(u) \cup \{L\}$
        compute score(L)
        $Q \leftarrow Q \cup \{L\}$
// Main loop
while $Q \neq \emptyset$ **do**
    extract $L = (v, t, C, \text{pred}, \text{start\_time})$ with maximum score from Q
    // Enforce 24-hour journey limit (1440 absolute minutes)
    if $t - \text{start\_time} > 1440$ **then**
        continue
    // Upper bound computation (Time-Budgeted)
    $b \leftarrow$ bucket(t)
    $U \leftarrow R(v, b) \setminus C$
    Sort U as a sequence $(c_1, c_2, \ldots, c_k)$ such that $MTT(c_1) \leq MTT(c_2) \leq \ldots \leq MTT(c_k)$
    time_spent $\leftarrow 0$
    UB $\leftarrow 0$
    remaining_time $\leftarrow 1440 - (t - \text{start\_time})$
    for $i = 1$ to $|U|$ **do**
        time_spent $\leftarrow$ time_spent $+ MTT(c_i)$
        if time_spent $\leq$ remaining_time **then**
            UB $\leftarrow$ UB $+ 1$
        else
            break
    // Record-based pruning
    if $|C| +$ UB $\leq$ record **then**
        continue

```
    // Update best label
    if best_label = null ∨ |C| > |best_label.C| then
        best_label ← L
        if |C| > record then
            break // early stopping: record beaten
    // Extend via scheduled edges
    for all (v, u, dep, arr) ∈ E_s do
        if dep ≥ t ∧ dep − start_time ≤ 1440 then
            t_new ← arr
            if t_new − start_time > 1440 then
                continue
            C' ← C ∪ {country(u)}
            L' ← (u, t_new, C', pred = L, start_time = start_time)

            if ∄L'' ∈ Labels(u) such that
                (L''.t ≤ t_new ∧ L''.C ⊇ C' ∧ (L''.t < t_new ∨ L''.C ≠ C')) then
                remove all L'' ∈ Labels(u) such that
                    (t_new ≤ L''.t ∧ C' ⊇ L''.C ∧ (t_new < L''.t ∨ C' ≠ L''.C))
                Labels(u) ← Labels(u) ∪ {L'}
                compute score(L')
                Q ← Q ∪ {L'}
    // Extend via transfers
    for all (v, w, Δ) ∈ E_t do
        t_new ← t + Δ
        if t_new − start_time > 1440 then
            continue
        L' ← (w, t_new, C, pred = L, start_time = start_time)

        if ∄L'' ∈ Labels(w) such that
            (L''.t ≤ t_new ∧ L''.C ⊇ C ∧ (L''.t < t_new ∨ L''.C ≠ C)) then
            remove all L'' ∈ Labels(w) such that
                (t_new ≤ L''.t ∧ C ⊇ L''.C ∧ (t_new < L''.t ∨ C ≠ L''.C))
            Labels(w) ← Labels(w) ∪ {L'}
            compute score(L')
            Q ← Q ∪ {L'}
// Reconstruct path
P ← empty sequence
L ← best_label
while L ≠ null do
    prepend L to P
    L ← L.pred
return P
```

A rapid, randomized greedy probe designed to reach the 24-hour mark in milliseconds. It utilizes heavily biased weights for unvisited countries and a Tabu list to prevent local loops.

Algorithm 7: Smart-Greedy Rollout Probe with Tabu List

Input:

- L_{start}. The starting label $(v, t, C, \text{pred}, \text{start\_time})$
- E_s. Set of scheduled edges $(v, u, \text{dep}, \text{arr})$
- E_t. Set of transfer edges (v, w, Δ)
- country(v). Country associated with node v

Output:

- L_{final}. The final label reached when the 24-hour clock expires

$L \leftarrow L_{\text{start}}$
RolloutVisitedNodes $\leftarrow \{L.\text{node}\}$ // Tabu list to prevent all local loops
// Repeat until no more valid moves within 24 hours
repeat
  $BestNodeArrivals \leftarrow$ Empty Map // Maps a node 'u' to the earliest arrival time 'tnew' and departure 'dep'
  // 1. Collect ONLY the earliest arrival for each neighboring node
  for all $(L.\text{node}, u, \text{dep}, \text{arr}) \in E_s$ **do**
    if $\text{dep} \geq L.t$ & $\text{arr} - L.\text{start\_time} \leq 1440$ **then**
      if $u \notin BestNodeArrivals$ **or** $\text{arr} < BestNodeArrivals[u].\text{arr}$ **then**
        $BestNodeArrivals[u] \leftarrow (\text{dep}, \text{arr}, \text{country}(u))$
  for all $(L.\text{node}, w, \Delta) \in E_t$ **do**
    $t_{\text{new}} \leftarrow L.t + \Delta$
    if $t_{\text{new}} - L.\text{start\_time} \leq 1440$ **then**
      if $w \notin BestNodeArrivals$ **or** $t_{\text{new}} < BestNodeArrivals[w].\text{arr}$ **then**
        // For transfers, dep and arr are both tnew
        $BestNodeArrivals[w] \leftarrow (t_{\text{new}}, t_{\text{new}}, \text{country}(w))$
  if $BestNodeArrivals$ is empty **then**
    break // No valid forward moves left
  // 2. Assign heavily biased weights to the deduplicated list
  $ValidSteps \leftarrow \emptyset$
  curr_country $\leftarrow$ country$(L.\text{node})$
  for all $u \in BestNodeArrivals$ **do**
    $(\text{dep}, \text{arr}, c_{\text{new}}) \leftarrow BestNodeArrivals[u]$
    $\Delta t \leftarrow \max(1, \text{arr} - L.t)$ // Total time consumed by this step
    if $c_{\text{new}} \notin L.C$ **then**
      // UNVISITED COUNTRY: Absolute highest priority.
      base_weight $\leftarrow 100,000$
    else if $c_{\text{new}} \neq$ curr_country **then**
      // TRANSIT VIA OTHER COUNTRY: Crucial for escaping geographical traps.
      base_weight $\leftarrow 1,000$
    else
      // DOMESTIC TRANSIT: Just used to get to hubs.
      base_weight $\leftarrow 1$
    weight $\leftarrow$ base_weight$/\Delta t$ // Linear time penalty
    // THE TABU PENALTY: Prevent wandering in local loops
    if $u \in$ RolloutVisitedNodes **then**
      weight $\leftarrow$ weight $\times 0.0001$
    $ValidSteps \leftarrow ValidSteps \cup \{(u, \text{arr}, c_{\text{new}}, \text{weight})\}$

39: | *// 3. Roll the weighted dice*
40: | $s_{\text{chosen}} \leftarrow \text{WeightedRandomChoice}(ValidSteps)$
41: | *// 4. Update State*
42: | $C' \leftarrow L.C \cup \{s_{\text{chosen}}.c_{\text{new}}\}$
43: | RolloutVisitedNodes $\leftarrow$ RolloutVisitedNodes $\cup \{s_{\text{chosen}}.u\}$
44: | $L \leftarrow (s_{\text{chosen}}.u, s_{\text{chosen}}.\text{arr}, C', \text{pred} = L, \text{start\_time} = L.\text{start\_time})$
45: **until** false
46: **return** L

Integrates Algorithm 7 into the main queue expansion, triggering the rapid rollouts every 1,000 iterations to attempt early global record updates.

Algorithm 8: Label-setting algorithm with time-budgeted UB and Monte-Carlo rollouts

---

Input:

- V. Set of nodes
- E_s. Set of scheduled edges $(v, u, \text{dep}, \text{arr})$
- E_t. Set of transfer edges (v, w, Δ)
- country(v). Country associated with node v
- $R(v, b)$. Precomputed reachable country sets
- $MTT(c)$. Precomputed minimum transit times for each country
- bucket(t). Time bucket mapping function
- record. Current record to beat (e.g., 13)
- rollout_freq. Number of iterations between rollouts (e.g., 1000)

Output:

- P. Sequence of labels representing a route within 24 hours

Initialize Labels$(v) \leftarrow \emptyset$ for all $v \in V$
Initialize priority queue $Q \leftarrow \emptyset$ // ordered by decreasing score
Initialize best_label $\leftarrow$ null
iters $\leftarrow 0$ // Iteration counter for rollouts
// Initialize labels at departures
for all $v \in V$ **do**
    for all scheduled edges $(v, u, \text{dep}, \text{arr}) \in E_s$ **do**
        $L \leftarrow (u, \text{arr}, \{\text{country}(u)\}, \text{pred} = \text{null}, \text{start\_time} = \text{dep})$
        Labels$(u) \leftarrow$ Labels$(u) \cup \{L\}$
        compute score(L)
        $Q \leftarrow Q \cup \{L\}$
// Main loop
while $Q \neq \emptyset$ **do**
    extract $L = (v, t, C, \text{pred}, \text{start\_time})$ with maximum score from Q
    // — NEW: MONTE CARLO ROLLOUT TRIGGER —
    iters $\leftarrow$ iters $+ 1$
    if iters mod rollout_freq $= 0$ **then**
        $L_{\text{rollout}} \leftarrow$ Algorithm 7$(L, E_s, E_t, \text{country})$
        if best_label $=$ null $\vee$ $|L_{\text{rollout}}.C| > |\text{best\_label}.C|$ **then**
            best_label $\leftarrow L_{\text{rollout}}$
            if $|\text{best\_label}.C| >$ record **then**
                break // early stopping: rollout found a record-breaking path
        // ——————————————————-
    // Enforce 24-hour journey limit (1440 absolute minutes)
    if $t -$ start_time > 1440 **then**
        continue
    // Upper bound computation (Time-Budgeted)
    $b \leftarrow$ bucket(t)
    $U \leftarrow R(v, b) \setminus C$
    Sort U as a sequence $(c_1, c_2, \ldots, c_k)$ such that $MTT(c_1) \leq MTT(c_2) \leq \ldots \leq MTT(c_k)$
    time_spent $\leftarrow 0$
    UB $\leftarrow 0$

```
    remaining_time ← 1440 − (t − start_time)
    for i = 1 to |U| do
        time_spent ← time_spent + MTT(c_i)
        if time_spent ≤ remaining_time then
            UB ← UB + 1
        else
            break
    // Record-based pruning
    if |C| + UB ≤ record then
        continue
    // Update best label from normal queue expansion
    if best_label = null ∨ |C| > |best_label.C| then
        best_label ← L
        if |C| > record then
            break // early stopping: record beaten
    // Extend via scheduled edges
    for all (v, u, dep, arr) ∈ E_s do
        if dep ≥ t  &  dep − start_time ≤ 1440 then
            t_new ← arr
            if t_new − start_time > 1440 then
                continue
            C' ← C ∪ {country(u)}
            L' ← (u, t_new, C', pred = L, start_time = start_time)

            if ∄L'' ∈ Labels(u) such that
                (L''.t ≤ t_new ∧ L''.C ⊇ C' ∧ (L''.t < t_new ∨ L''.C ≠ C')) then
                remove all L'' ∈ Labels(u) such that
                    (t_new ≤ L''.t ∧ C' ⊇ L''.C ∧ (t_new < L''.t ∨ C' ≠ L''.C))
                Labels(u) ← Labels(u) ∪ {L'}
                compute score(L')
                Q ← Q ∪ {L'}
    // Extend via transfers
    for all (v, w, Δ) ∈ E_t do
        t_new ← t + Δ
        if t_new − start_time > 1440 then
            continue
        L' ← (w, t_new, C, pred = L, start_time = start_time)

        if ∄L'' ∈ Labels(w) such that
            (L''.t ≤ t_new ∧ L''.C ⊇ C ∧ (L''.t < t_new ∨ L''.C ≠ C)) then
            remove all L'' ∈ Labels(w) such that
                (t_new ≤ L''.t ∧ C ⊇ L''.C ∧ (t_new < L''.t ∨ C ≠ L''.C))
            Labels(w) ← Labels(w) ∪ {L'}
            compute score(L')
            Q ← Q ∪ {L'}
// Reconstruct path
P ← empty sequence
L ← best_label
while L ≠ null do
    prepend L to P
    L ← L.pred
return P
```

Introduces a GlobalMinTime constant to aggressively prune labels that mathematically lack the physical time required to bridge the gap between their current country count and the global record.

Algorithm 9: Label-setting with Global Hard Floor, Time-Budgeted UB, and Monte-Carlo Rollouts

---

Input:

- V. Set of nodes
- E_s. Set of scheduled edges $(v, u, \text{dep}, \text{arr})$
- E_t. Set of transfer edges (v, w, Δ)
- country(v). Country associated with node v
- $R(v, b)$. Precomputed reachable country sets
- $MTT(c)$. Precomputed minimum transit times for each country
- bucket(t). Time bucket mapping function
- record. Current record to beat (e.g., 13). Static.
- rollout_freq. Number of iterations between rollouts (e.g., 1000)
- GlobalMinTime. Minimum realistic time to visit a country (e.g., 30 mins)

Output:

- P. Sequence of labels representing a route within 24 hours

Initialize Labels$(v) \leftarrow \emptyset$ for all $v \in V$
Initialize priority queue $Q \leftarrow \emptyset$ // ordered by decreasing score
Initialize best label $\leftarrow$ null
iters $\leftarrow 0$ // Iteration counter for rollouts
// Initialize labels at departures
for all $v \in V$ **do**
    for all scheduled edges $(v, u, \text{dep}, \text{arr}) \in E_s$ **do**
        $L \leftarrow (u, \text{arr}, \{\text{country}(u)\}, \text{pred} = \text{null}, \text{start\_time} = \text{dep})$
        Labels$(u) \leftarrow$ Labels$(u) \cup \{L\}$
        compute score(L)
        $Q \leftarrow Q \cup \{L\}$
// Main loop
while $Q \neq \emptyset$ **do**
    extract $L = (v, t, C, \text{pred}, \text{start\_time})$ with maximum score from Q
    // — GLOBAL HARD FLOOR PRUNING —
    needed $\leftarrow (\text{record} + 1) - |C|$
    remaining_time $\leftarrow 1440 - (t - \text{start\_time})$
    // If we need countries but lack the physical time (e.g., ¡ 30 mins per country), kill it.
    if needed $> 0 \wedge$ remaining_time $<$ (needed $\times$ GlobalMinTime) **then**
        continue
    // ————————————————
    // — MONTE CARLO ROLLOUT TRIGGER —
    iters $\leftarrow$ iters $+ 1$
    if iters mod rollout_freq $= 0$ **then**
        // Unconditional rollout for any label that survived Hard Floor
        $L_{\text{rollout}} \leftarrow$ Algorithm 7$(L, E_s, E_t, \text{country})$
        if best label $=$ null $\vee\ |L_{\text{rollout}}.C| > |\text{best label}.C|$ **then**
            best label $\leftarrow L_{\text{rollout}}$
        // Early stopping if rollout beats the static record
        if $|\text{best label}.C| >$ record **then**
            break
    // ————————————————-

```
    // Enforce 24-hour journey limit (1440 absolute minutes)
    if remaining_time < 0 then
        continue
    // Upper bound computation (Time-Budgeted)
    b ← bucket(t)
    U ← R(v, b) \ C
    Sort U as a sequence (c_1, c_2, ..., c_k) such that MTT(c_1) ≤ MTT(c_2) ≤ ... ≤ MTT(c_k)
    time_spent ← 0
    UB ← 0
    for i = 1 to |U| do
        time_spent ← time_spent + MTT(c_i)
        if time_spent ≤ remaining_time then
            UB ← UB + 1
        else
            break
    // Record-based pruning (Using static record)
    if |C| + UB ≤ record then
        continue
    // Update best label from normal queue expansion
    if best label = null ∨ |C| > |best label.C| then
        best label ← L
    // Early stopping if normal expansion beats the static record
    if |C| > record then
        break
    // Extend via scheduled edges
    for all (v, u, dep, arr) ∈ E_s do
        if dep ≥ t  &  dep − start_time ≤ 1440 then
            t_new ← arr
            if t_new − start_time > 1440 then
                continue
            C' ← C ∪ {country(u)}
            L' ← (u, t_new, C', pred = L, start_time = start_time)
            if ∄L'' ∈ Labels(u) such that
                (L''.t ≤ t_new ∧ L''.C ⊇ C' ∧ (L''.t < t_new ∨ L''.C ≠ C')) then
                remove all L'' ∈ Labels(u) such that
                    (t_new ≤ L''.t ∧ C' ⊇ L''.C ∧ (t_new < L''.t ∨ C' ≠ L''.C))
                Labels(u) ← Labels(u) ∪ {L'}
                compute score(L')
                Q ← Q ∪ {L'}
    // Extend via transfers
    for all (v, w, Δ) ∈ E_t do
        t_new ← t + Δ
        if t_new − start_time > 1440 then
            continue
        L' ← (w, t_new, C, pred = L, start_time = start_time)
        if ∄L'' ∈ Labels(w) such that
            (L''.t ≤ t_new ∧ L''.C ⊇ C ∧ (L''.t < t_new ∨ L''.C ≠ C)) then
            remove all L'' ∈ Labels(w) such that
                (t_new ≤ L''.t ∧ C ⊇ L''.C ∧ (t_new < L''.t ∨ C ≠ L''.C))
            Labels(w) ← Labels(w) ∪ {L'}
```

```
79:         compute score(L′)
80:         Q ← Q ∪ {L′}
81: // Reconstruct path
82: P ← empty sequence
83: L ← best label
84: while L ≠ null do
85:     prepend L to P
86:     L ← L.pred
87: return P
```

The first half of the bi-directional pivot. It executes the time-budgeted, hard-floor search strictly forward in time, terminating at the 14-hour mark to avoid nocturnal graph sparsity.

Algorithm 10: Bounded Forward Half-Search

Input:

- V. Set of nodes
- E_s. Set of scheduled edges $(v, u, \text{dep}, \text{arr})$
- E_t. Set of transfer edges (v, w, Δ)
- country(v). Country associated with node v
- $R(v, b)$. Precomputed reachable country sets (from Algorithm 3)
- $MTT(c)$. Precomputed minimum transit times (from Algorithm 5)
- bucket(t). Time bucket mapping function
- MaxDuration. Maximum allowed elapsed time for the half-search (e.g., 840 mins / 14 hours)
- MinCountries. Minimum unique countries required to save the path (e.g., 6)
- GlobalMinTime. Minimum realistic time to visit a country (e.g., 25 mins)

Output:

- Labels$_F$. Map of nodes to a set of Pareto-optimal forward labels

Initialize Labels$_F(v) \leftarrow \emptyset$ for all $v \in V$
Initialize priority queue $Q \leftarrow \emptyset$ // ordered by decreasing score
// Initialize labels at departures
for all $v \in V$ **do**
    for all scheduled edges $(v, u, \text{dep}, \text{arr}) \in E_s$ **do**
        $L \leftarrow (\text{node} = u, t = \text{arr}, C = \{\text{country}(u)\}, \text{pred} = \text{null}, \text{start\_time} = \text{arr})$
        Labels$_F(u) \leftarrow$ Labels$_F(u) \cup \{L\}$
        compute score$(L) \leftarrow (|L.C| \times 10000) - L.t$
        $Q \leftarrow Q \cup \{L\}$
// Main loop
while $Q \neq \emptyset$ **do**
    extract $L = (v, t, C, \text{pred}, \text{start\_time})$ with maximum score from Q
    elapsed $\leftarrow t -$ start_time
    remaining_half_time $\leftarrow$ MaxDuration $-$ elapsed
    // — 1. GLOBAL HARD FLOOR PRUNING (Half-Search Variant) —
    needed $\leftarrow$ MinCountries $- |C|$
    // If the path physically cannot even reach the minimum required countries in the allowed 14 hours, kill it.
    if needed $> 0 \wedge$ remaining_half_time $<$ (needed $\times$ GlobalMinTime) **then**
        continue
    // — 2. TIME-BUDGETED UB PRUNING (Half-Search Variant) —
    if needed > 0 **then**
        $b \leftarrow$ bucket(t)
        $U \leftarrow R(v, b) \setminus C$
        Sort U as a sequence $(c_1, c_2, \ldots, c_k)$ such that $MTT(c_1) \leq MTT(c_2) \leq \ldots \leq MTT(c_k)$
        time_spent $\leftarrow 0$
        UB $\leftarrow 0$
        for $i = 1$ to $|U|$ **do**
            time_spent $\leftarrow$ time_spent $+ MTT(c_i)$
            if time_spent $\leq$ remaining_half_time **then**
                UB $\leftarrow$ UB $+ 1$
            else

```
            break
        // If current countries + theoretically reachable countries ¡ minimum acceptable, kill it.
        if |C| + UB < MinCountries then
            continue
    // — 3. EXTEND VIA SCHEDULED EDGES —
    for all (v, u, dep, arr) ∈ E_s do
        if dep ≥ t then
            new_elapsed ← arr − start_time
            // Enforce the 14-hour cut-off limit
            if new_elapsed ≤ MaxDuration then
                C' ← C ∪ {country(u)}
                L' ← (u, arr, C', pred = L, start_time = start_time)

                // Strict Pareto Dominance
                if ∄L'' ∈ Labels_F(u) such that
                (L''.t ≤ arr ∧ L''.C ⊇ C' ∧ (L''.t < arr ∨ L''.C ≠ C')) then
                remove all L'' ∈ Labels_F(u) such that
                    (arr ≤ L''.t ∧ C' ⊇ L''.C ∧ (arr < L''.t ∨ C' ≠ L''.C))
                Labels_F(u) ← Labels_F(u) ∪ {L'}
                compute score(L')
                Q ← Q ∪ {L'}
    // — 4. EXTEND VIA TRANSFERS —
    for all (v, w, Δ) ∈ E_t do
        t_new ← t + Δ
        new_elapsed ← t_new − start_time
        if new_elapsed ≤ MaxDuration then
            L' ← (w, t_new, C, pred = L, start_time = start_time)

            // Strict Pareto Dominance
            if ∄L'' ∈ Labels_F(w) such that
            (L''.t ≤ t_new ∧ L''.C ⊇ C ∧ (L''.t < t_new ∨ L''.C ≠ C)) then
            remove all L'' ∈ Labels_F(w) such that
                (t_new ≤ L''.t ∧ C ⊇ L''.C ∧ (t_new < L''.t ∨ C ≠ L''.C))
            Labels_F(w) ← Labels_F(w) ∪ {L'}
            compute score(L')
            Q ← Q ∪ {L'}
// — 5. CLEANUP —
// Purge any stored labels that survived Pareto dominance but failed to hit the final country threshold.
for all v ∈ V do
    for all L ∈ Labels_F(v) do
        if |L.C| < MinCountries then
            remove L from Labels_F(v)
return Labels_F
```

The second half of the bi-directional pivot. It evaluates the network in reverse, tracking inbound scheduled edges backward from the end of the 24-hour window, terminating at the 14-hour mark.

Algorithm 11: Bounded Backward Half-Search

Input:

- V. Set of nodes
- $E_{s,\text{in}}(v)$. Set of scheduled INBOUND edges to v: $(w, v, \text{dep}, \text{arr})$
- $E_{t,\text{in}}(v)$. Set of INBOUND transfer edges to v: (w, v, Δ)
- country(v). Country associated with node v
- $R_{\text{back}}(v, b)$. Precomputed reverse reachable country sets
- $MTT(c)$. Precomputed minimum transit times
- bucket(t). Time bucket mapping function
- MaxDuration. Maximum allowed elapsed time for the half-search (e.g., 840 mins)
- MinCountries. Minimum unique countries required to save the path (e.g., 6)
- GlobalMinTime. Minimum realistic time to visit a country (e.g., 25 mins)

Output:

- Labels$_B$. Map of nodes to a set of Pareto-optimal backward labels

Initialize Labels$_B(v) \leftarrow \emptyset$ for all $v \in V$
Initialize priority queue $Q \leftarrow \emptyset$ // ordered by decreasing score
// Initialize labels at arrivals (End of the journey)
for all $v \in V$ **do**
    for all scheduled edges $(w, v, \text{dep}, \text{arr}) \in E_{s,\text{in}}(v)$ **do**
        // We initialize the state sitting at the arrival node 'v', having just finished our journey
        $L \leftarrow (\text{node} = v, t = \text{arr}, C = \{\text{country}(v)\}, \text{succ} = \text{null}, \text{end\_time} = \text{arr})$
        Labels$_B(v) \leftarrow$ Labels$_B(v) \cup \{L\}$
        // Score favors paths with MORE countries and LATER departure times (less elapsed time)
        compute score$(L) \leftarrow (|L.C| \times 10000) + L.t$
        $Q \leftarrow Q \cup \{L\}$
// Main loop
while $Q \neq \emptyset$ **do**
    extract $L = (v, t, C, \text{succ}, \text{end\_time})$ with maximum score from Q
    elapsed $\leftarrow$ end_time $-$ t
    remaining_half_time $\leftarrow$ MaxDuration $-$ elapsed
    // — 1. GLOBAL HARD FLOOR PRUNING (Backward Variant) —
    needed $\leftarrow$ MinCountries $- |C|$
    if needed $> 0 \wedge$ remaining_half_time $<$ (needed $\times$ GlobalMinTime) **then**
        continue
    // — 2. TIME-BUDGETED UB PRUNING (Backward Variant) —
    if needed > 0 **then**
        $b \leftarrow$ bucket(t)
        $U \leftarrow R_{\text{back}}(v, b) \setminus C$
        Sort U as a sequence $(c_1, c_2, \ldots, c_k)$ such that $MTT(c_1) \leq MTT(c_2) \leq \ldots \leq MTT(c_k)$
        time_spent $\leftarrow 0$
        UB $\leftarrow 0$
        for $i = 1$ to $|U|$ **do**
            time_spent $\leftarrow$ time_spent $+ MTT(c_i)$
            if time_spent $\leq$ remaining_half_time **then**
                UB $\leftarrow$ UB $+ 1$
            else

```
        break
    if |C| + UB < MinCountries then
        continue
// — 3. EXTEND BACKWARD VIA SCHEDULED EDGES —
// We look for trains that ARRIVED at our current node BEFORE our current time 't'
for all (w, v, dep, arr) ∈ E_{s,in}(v) do
    if arr ≤ t then
        new_elapsed ← end_time − dep
        if new_elapsed ≤ MaxDuration then
            C' ← C ∪ {country(w)}
            // The new state is sitting at node 'w' at time 'dep'
            L' ← (w, dep, C', succ = L, end_time = end_time)

            // STRICT BACKWARD PARETO DOMINANCE: t must be ¿= to be better
            if ∄L'' ∈ Labels_B(w) such that
                (L''.t ≥ dep ∧ L''.C ⊇ C' ∧ (L''.t > dep ∨ L''.C ≠ C')) then
                remove all L'' ∈ Labels_B(w) such that
                    (dep ≥ L''.t ∧ C' ⊇ L''.C ∧ (dep > L''.t ∨ C' ≠ L''.C))
                Labels_B(w) ← Labels_B(w) ∪ {L'}
                compute score(L')
                Q ← Q ∪ {L'}
// — 4. EXTEND BACKWARD VIA TRANSFERS —
for all (w, v, Δ) ∈ E_{t,in}(v) do
    t_new ← t − Δ // Subtract walking time to find when we needed to start walking
    new_elapsed ← end_time − t_new
    if new_elapsed ≤ MaxDuration then
        L' ← (w, t_new, C, succ = L, end_time = end_time)

        // Strict Backward Pareto Dominance
        if ∄L'' ∈ Labels_B(w) such that
            (L''.t ≥ t_new ∧ L''.C ⊇ C ∧ (L''.t > t_new ∨ L''.C ≠ C)) then
            remove all L'' ∈ Labels_B(w) such that
                (t_new ≥ L''.t ∧ C ⊇ L''.C ∧ (t_new > L''.t ∨ C ≠ L''.C))
            Labels_B(w) ← Labels_B(w) ∪ {L'}
            compute score(L')
            Q ← Q ∪ {L'}
// — 5. CLEANUP —
for all v ∈ V do
    for all L ∈ Labels_B(v) do
        if |L.C| < MinCountries then
            remove L from Labels_B(v)
return Labels_B
```

Evaluates the intersection of the forward and backward half-searches. It validates temporal continuity and geographic unions to reconstruct the globally optimal 24-hour path.

Algorithm 12: Bi-directional Stitcher

Input:

- V. Set of nodes
- Labels_F. Map of Pareto-optimal forward labels from Algorithm 10
- Labels_B. Map of Pareto-optimal backward labels from Algorithm 11
- Record. The global static record to beat (e.g., 13)
- MinWait. Minimum connection time required at a station (e.g., 5 or 10 mins)

Output:

- P_{best}. The reconstructed sequence of labels representing the best route, or null if no route beats the record.

```
best_route ← null
best_countries ← ∅
best_duration ← ∞
// 1. Scan the intersection of nodes
for all v ∈ V do
    if Labels_F(v) = ∅ ∨ Labels_B(v) = ∅ then
        continue // Node was not reached by both halves
    // 2. Iterate through all combinations of forward and backward labels at node 'v'
    for all L_f ∈ Labels_F(v) do
        for all L_b ∈ Labels_B(v) do
            // — CHECK A: TIME CONTINUITY —
            // Does the forward path arrive in time to catch the backward path's departure?
            wait_time ← L_b.t − L_f.t
            if wait_time < MinWait then
                continue
            // — CHECK B: 24-HOUR CONSTRAINT —
            // Is the total combined elapsed time strictly within 24 hours?
            total_duration ← L_b.end_time − L_f.start_time
            if total_duration > 1440 then
                continue
            // — CHECK C: GEOGRAPHIC UNION —
            // Merge the two sets of countries (this naturally deduplicates visited countries)
            C_combined ← L_f.C ∪ L_b.C
            // — EVALUATE AND UPDATE —
            // We strictly want to beat the record. If it ties the current best, we can tie-break by duration.
            if |C_combined| > Record ∨ (|C_combined| = Record ∧ total_duration < best_duration) then
                Record ← |C_combined| // Dynamically raise the bar for the rest of the loop
                best_countries ← C_combined
                best_duration ← total_duration
                // 3. Reconstruct the stitched path
                best_route ← ReconstructStitchedPath(L_f, L_b)
return best_route
// Auxiliary Function: ReconstructStitchedPath(L_f, L_b)

P_forward ← empty sequence
curr ← L_f
// Trace forward path backwards to the start
```

```
37: while curr ≠ null do
38:     prepend curr to P_forward
39:     curr ← curr.pred
40: P_backward ← empty sequence
41: // Note: We skip the first node of L_b to avoid duplicating the meeting station 'v'
42: curr ← L_b.succ
43: // Trace backward path forwards to the destination
44: while curr ≠ null do
45:     append curr to P_backward
46:     curr ← curr.succ
47: return P_forward concatenated with P_backward
```

C

Temporal and Toponymic Normalization Dictionaries

This appendix details the explicit hardcoded dictionaries implemented within the data engineering pipeline to resolve upstream API inconsistencies, enforce deterministic UTC synchronization, and correct erroneous geographical assignments at complex European border crossings.

C.1 UTC Timezone Airport Overrides

While standard timezone synchronization relied on ISO alpha-2 country codes, outermost regions and non-contiguous territories required explicit IATA code overrides to prevent negative time-travel artifacts in the Directed Acyclic Graph. The following dictionary defines the specific offsets (in minutes) applied relative to UTC+0:

```
AIRPORT_OVERRIDES =
    # UTC -1 (Azores)
    "PDL": -60, "TER": -60, "FLW": -60, "CVU": -60, "HOR": -60,
    "PIX": -60, "GRW": -60, "SJZ": -60, "SMA": -60,
    # UTC +0 (Canary Islands, Madeira, Faroe Islands,
    # Crown Dependencies, Iceland)
    "LPA": 0, "TFS": 0, "TFN": 0, "ACE": 0, "FUE": 0, "SPC": 0,
    "VDE": 0, "GMZ": 0, "FNC": 0, "PXO": 0, "FAE": 0, "JER": 0,
    "GCI": 0, "IOM": 0, "ACI": 0, "AEY": 0, "EGS": 0, "GRY": 0,
    "IFJ": 0, "RKV": 0, "VPN": 0, "KEF": 0, "HFN": 0, "BIU": 0,
    # UTC +2 (Moldova)
    "RMO": 120, "KIV": 120,
    # UTC +3 (Turkey, Belarus, Western Russia)
    "IST": 180, "SAW": 180, "AYT": 180, "ESB": 180, "ADB": 180,
    "DLM": 180, "BJV": 180, "TZX": 180, "GZP": 180, "TEQ": 180,
    "SIP": 180, "MSQ": 180, "MVQ": 180, "BQT": 180, "GME": 180,
    "LED": 180, "SVO": 180, "DME": 180, "VKO": 180, "ZIA": 180,
    "AER": 180, "AAQ": 180, "KRR": 180, "ROV": 180, "VOZ": 180,
    # UTC +4 to +7 (Eastern Russian Hubs)
    "KUF": 240, "UFA": 300, "SVX": 300, "PEE": 300, "OVB": 420,
    "NOZ": 420, "KJA": 420
```

C.2 Geopolitical Standardization and Border Corrections

To ensure the multi-modal routing algorithm properly merged flight and rail data without fragmenting sovereign territories, geopolitical names were mapped to a unified standard. Additionally, stations situated near complex international borders whose administrative country differed from their linguistic profile were forcefully corrected using the following mappings.

Geopolitical Standardization:

```
SPECIAL_COUNTRY_NAME_MAP =
    "england": "GB",
    "scotland": "GB",
    "wales": "GB",
    "northern ireland": "GB",
    "united kingdom": "England", # Unified to match Deutsche Bahn
    "moldova, republic of": "Moldova"
```

Critical Border Station Corrections (Subset):

```
STATION_CORRECTIONS =
    "Genève, Mont-Blanc": "CH",    # Switzerland (Not France)
    "St-Léonard": "CH",            # Switzerland (Not France)
    "Bratislava hl.st.": "SK",       # Slovakia (Not Czechia)
    "Baden b.Wien": "AT",            # Austria (Not Germany)
    "Salzburg Liefering": "AT",      # Austria (Not Germany)
    "Ponte Gardena-Laion": "IT",     # Italy (Not Austria)
    "Domodossola": "IT",             # Italy (Border station)
    "Rybnik(CZ)": "PL",              # Poland (Not Czechia despite suffix)
    "Koebenhavns Lufthavn st": "DK",# Denmark
    "Visby st": "SE",                # Sweden (Not Denmark)
    "Komarom": "HU",                 # Hungary (Slovak side is Komarno)
    "Schaanwald, Zollamt": "LI"      # Liechtenstein (Not Switzerland)
```